# DSA Midterm Exam Master Cheatsheet (Java Implementation Templates)

## SECTION 1: JAVA SURVIVAL BLUEPRINT (PYTHON/R TO JAVA BRIDGE)

### 1.1 Minimal Exam Java Class & Method Anatomy

In Java, every variable and function must live inside a `class`. The entry point for execution is `main`.

```
// Template: Minimal runnable standalone program
public class Solution {
    public static void main(String[] args) {
        // Quick variable definitions
        int n = 10;
        int[] arr = {5, 2, 8, 1, 9};

        // Calling static helper methods
        bubbleSort(arr);

        // Printing output
        System.out.println(java.util.Arrays.toString(arr));
    }

    // Method template: public static <return_type> <name>(<params>)
    public static void bubbleSort(int[] a) {
        // Implementation here
    }
}
```

### 1.2 Primitives vs. References & Memory Model

- **Primitive Types**: `int`, `long`, `double`, `boolean`, `char`. Stored directly on the Call Stack. Fast, pass-by-value.
- **Reference Types**: Arrays (`int[]`), Objects (`Node`, `String`, `Stack`). Variable stores a 64-bit memory address (pointer) pointing to data on the Heap.
- **The Equality Trap (`==` vs `.equals()`)**:
  - `a == b` for primitives compares actual values (`5 == 5` is `true`).
  - `obj1 == obj2` for objects compares memory addresses (identity), NOT contents!
  - `str1.equals(str2)` compares string contents. In an exam: use `==` for `int` keys, `.equals()` for `String` objects.

### 1.3 Array Mechanics & Memory Contiguity

Arrays in Java are fixed-size, 0-indexed, contiguous memory blocks.

```
// 1. Allocation
int[] arr = new int[5];              // Initialized to all 0s: [0, 0, 0, 0, 0]
int[] preset = {10, 20, 30, 40};      // Size is fixed at 4
int len = preset.length;             // .length is a field, NOT a function ()

// 2. Traversal patterns
for (int i = 0; i < arr.length; i++) { /* Standard forward scan */ }
for (int i = arr.length - 1; i >= 0; i--) { /* Backward scan */ }
for (int val : preset) { /* Enhanced for-each (read-only) */ }

// 3. Array Copying & Resizing
int[] copy = new int[arr.length * 2];
System.arraycopy(arr, 0, copy, 0, arr.length); // Ultra-fast O(N) memory copy
int[] cloned = arr.clone();                    // Shallow copy

// 4. Two-Dimensional Arrays (Matrix)
int rows = 3, cols = 4;
int[][] matrix = new int[rows][cols];
int numRows = matrix.length;
int numCols = matrix[0].length;
```

### 1.4 Essential Java OOP: Constructors, Getters, Setters & Encapsulation

While DSA exams focus on algorithms, exam prompts frequently define classes with `private` fields requiring explicit constructors, getters, and setters.

```
// Template: Canonical Java Class with Complete Encapsulation
public class DataRecord {
    // 1. Private fields (Encapsulation: hide internal state)
    private int id;
    private String name;
    private double score;

    // 2. Default No-Arg Constructor (initializes to sensible defaults)
    public DataRecord() {
        this.id = 0;
        this.name = "";
        this.score = 0.0;
    }

    // 3. Parameterized Constructor (this.field = param)
    public DataRecord(int id, String name, double score) {
        this.id = id;
        this.name = name;
        this.score = score;
    }

    // 4. Getters (Accessor: public <Type> get<Field>())
    public int getId()        { return this.id; }
    public String getName()   { return this.name; }
    public double getScore()  { return this.score; }

    // 5. Setters (Mutator: public void set<Field>(<Type> param))
    public void setId(int id) {
        if (id < 0) throw new IllegalArgumentException("ID must be positive");
        this.id = id;
    }
    public void setName(String name)   { this.name = name; }
    public void setScore(double score) { this.score = score; }

    // 6. toString() for easy debugging / printing
    public String toString() {
        return "DataRecord{id=" + id + ", name='" + name + "', score=" + score + "}";
    }
}
```

**Ready-to-Use Node Templates with Getters & Setters —**

```
// Singly Linked List Node
public class SNode {
    private int key;
    private SNode next;

    public SNode() { this(0, null); }
    public SNode(int key) { this(key, null); }
    public SNode(int key, SNode next) {
        this.key = key;
        this.next = next;
    }

    public int getKey()          { return this.key; }
    public void setKey(int k)    { this.key = k; }
    public SNode getNext()       { return this.next; }
    public void setNext(SNode n) { this.next = n; }
}

// Doubly Linked List Node
public class DNode {
    private int key;
    private DNode next;
    private DNode previous;

    public DNode(int key) {
        this.key = key;
        this.next = null;
        this.previous = null;
    }

    public int getKey()              { return this.key; }
    public void setKey(int k)        { this.key = k; }
    public DNode getNext()           { return this.next; }
    public void setNext(DNode n)     { this.next = n; }
    public DNode getPrevious()       { return this.previous; }
    public void setPrevious(DNode p) { this.previous = p; }
}
```

### 1.5 Common Exam Bugs & Defensive Guard Rails

1. **`NullPointerException` (NPE)**: Attempting to access fields on `null` (e.g., `current.next` when `current == null`). Always guard: `while (current != null && current.next != null)`.
2. **`ArrayIndexOutOfBoundsException`**: Accessing `arr[-1]` or `arr[arr.length]`. Valid indices: `0 <= idx < arr.length`.
3. **Integer Division Truncation**: `5 / 2` evaluates to `2`! For floating point, cast at least one operand: `(double) 5 / 2 == 2.5`.
4. **Binary Search Midpoint Overflow**: `int mid = (low + high) / 2;` can overflow 32-bit signed int if `low + high > 2^31 - 1`. Always write: `int mid = low + (high - low) / 2;` or `int mid = (low + high) >>> 1;`.
5. **Exam Rounding Convention**: In IT013IU exams, "rounding up" convention is used: `ceil(7 / 2.0) = 4`. In Java: `(a + b - 1) / b` computes ceiling of integer division `a / b`.

### 1.6 Standard Java Collections Cheat Sheet (If Standard Libs Permitted)

```
// Dynamic Array: ArrayList
java.util.ArrayList<Integer> list = new java.util.ArrayList<>();
list.add(10);           // O(1) amortized
int val = list.get(0);  // O(1)
list.set(0, 99);        // O(1)
list.remove(list.size() - 1); // O(1) tail removal
int sz = list.size();

// Stack (LIFO)
java.util.Stack<Integer> stack = new java.util.Stack<>();
stack.push(10);         // O(1)
int top = stack.pop();  // O(1) removes and returns
int peek = stack.peek();// O(1) inspects without removing
boolean empty = stack.isEmpty();

// FIFO Queue (Using LinkedList)
java.util.Queue<Integer> q = new java.util.LinkedList<>();
q.add(10);              // enqueue (throws on full)
q.offer(10);            // enqueue (returns false on full)
int front = q.poll();   // dequeue (returns null if empty)
int peekQ = q.peek();   // inspect front
int qSize = q.size();

// Priority Queue (Binary Heap)
// Default: MIN-HEAP (smallest item at top)
java.util.PriorityQueue<Integer> minPQ = new java.util.PriorityQueue<>();
minPQ.add(20); minPQ.add(5); minPQ.add(15);
int smallest = minPQ.poll(); // Returns 5

// MAX-HEAP (largest item at top)
java.util.PriorityQueue<Integer> maxPQ = new java.util.PriorityQueue<>(java.util.Collections.reverseOrder());
maxPQ.add(20); maxPQ.add(5); maxPQ.add(15);
int largest = maxPQ.poll();  // Returns 20
```

## SECTION 2: ASYMPTOTIC ANALYSIS & BIG-O DECISION MATRIX

### 2.1 Formal Definitions & Growth Orders

- **Big-O ($O(g(n))$)**: Asymptotic UPPER bound. $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. (Guaranteed worst-case performance).
- **Big-Omega ($\Omega(g(n))$)**: Asymptotic LOWER bound. $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$. (Best-case baseline).
- **Big-Theta ($\Theta(g(n))$)**: Asymptotic TIGHT bound. $c_1 g(n) \leq f(n) \leq c_2 g(n)$. Both upper and lower bound match.
- **Hierarchy of Growth Rates**:
$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

### 2.2 Loop Analysis Rules

1. **Single Loop (Linear $O(n)$)**: `for (int i = 0; i < n; i += c)` $\Longrightarrow O(n)$.
2. **Halving / Doubling Loop (Logarithmic $O(\log n)$)**: `for (int i = 1; i < n; i *= 2)` or `for (int i = n; i > 0; i /= 2)` $\Longrightarrow O(\log n)$.
3. **Nested Loops (Quadratic $O(n^2)$)**:
   - Dependent inner loop: `for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++)`
   - Total executions: $\sum_{i=0}^{n-1}(n-1-i) = \frac{n(n-1)}{2} = \frac{n^2-n}{2} \in \Theta(n^2)$.
4. **Divide and Conquer (Merge Sort $O(n \log n)$)**:
   - Recurrence: $T(n) = 2T(n/2) + O(n)$.
   - By Master Theorem: $a = 2, b = 2, d = 1 \Longrightarrow \log_b a = \log_2 2 = 1 = d \Longrightarrow T(n) \in \Theta(n \log n)$.

### 2.3 Comprehensive Data Structures & Algorithms Complexity Matrix

| Data Structure / Operation | Access (Index) | Search (Value) | Insertion (Head) | Insertion (Tail) | Deletion (Head) | Deletion (Tail) |
|---|---|---|---|---|---|---|
| **Array (Static)** | $O(1)$ | $O(n)$ | $O(n)$ shift | $O(1)$ if room | $O(n)$ shift | $O(1)$ |
| **Dynamic Array (ArrayList)** | $O(1)$ | $O(n)$ | $O(n)$ shift | $O(1)$ amortized | $O(n)$ shift | $O(1)$ |
| **Singly Linked List (no tail)** | $O(n)$ | $O(n)$ | $O(1)$ | $O(n)$ | $O(1)$ | $O(n)$ |
| **Singly Linked List (with tail)** | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ | $O(1)$ | $O(n)$ need prev |
| **Doubly Linked List (with tail)** | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ | $O(1)$ | $O(1)$ |
| **Stack (Array or Linked)** | — | $O(n)$ | $O(1)$ push | — | $O(1)$ pop | — |
| **FIFO Queue (Circular Array/Linked)** | — | $O(n)$ | — | $O(1)$ enqueue | $O(1)$ dequeue | — |
| **Deque (Double-Ended Queue)** | — | $O(n)$ | $O(1)$ push-Front | $O(1)$ pushRear | $O(1)$ popFront | $O(1)$ popRear |
| **Priority Queue (Binary Heap)** | $O(1)$ peek min/max | $O(n)$ | $O(\log n)$ insert | $O(\log n)$ insert | $O(\log n)$ extract | — |

| Sorting Algorithm | Best Time | Average Time | Worst Time | Auxiliary Space | Stable? | Comparisons (Avg) | Swaps / Shifts (Avg) |
|---|---|---|---|---|---|---|---|
| **Bubble Sort** | $O(n)$ flag | $O(n^2)$ | $O(n^2)$ | $O(1)$ | **YES** | $\approx n^2/2$ | $\approx 3n^2/4$ writes |
| **Selection Sort** | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | **NO** | Exactly $\frac{n(n-1)}{2}$ | At most $n-1$ swaps |
| **Insertion Sort** | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | **YES** | $\approx n^2/4$ | $\approx n^2/4$ shifts |
| **Merge Sort** | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ | **YES** | $\approx n \log_2 n$ | $O(n \log n)$ copies |

## SECTION 2.5: IN-DEPTH ALGORITHM THEORY, MECHANICS & COMPARISONS

### 2.5.1 Comprehensive Algorithm Mechanics ("How They Work")

**1. Linear Search (Unsorted Array) —**
- **Mechanical Principle**: Sequential brute-force scan from index `0` to `n - 1`.
- **How it Works**: Compares `arr[i] == target` at each step. If a match is found, immediately returns the index `i`. If the loop finishes without finding `target`, returns `-1`.
- **Complexity**: Best $O(1)$ (target is at index 0); Worst $O(n)$ ($n$ comparisons if target is at end or absent); Average $n/2 \in O(n)$.
- **Key Invariant**: Requires ZERO structural prerequisites. Works on unsorted, duplicate-heavy, and unindexed streams.

**2. Optimized Linear Search Variants —**
- **Variant A: Sentinel Linear Search**:
  - *Mechanism*: Eliminates the loop boundary condition `i < n` on every cycle. It temporarily sets `arr[n - 1] = target`. The loop simply runs `while (arr[i] != target) i++;`.
  - *Why it's Faster*: Cuts CPU branch predictions in half (1 comparison per loop iteration instead of 2: checking `i < n` AND checking `arr[i] == target`). Still $O(n)$, but with half the machine cycle overhead.
- **Variant B: Sorted Linear Search (Early-Exit)**:
  - *Mechanism*: Assumes array is sorted. Scans forward until `arr[i] >= target`. If `arr[i] == target`, returns `i`; if `arr[i] > target`, stops immediately and returns `-1`.
  - *Advantage*: Missing elements are discovered in $n/2$ steps on average instead of $n$. Still $O(n)$ worst-case.
- **Variant C: Self-Organizing Linear Search (Move-To-Front / Transposition)**:
  - *Mechanism*: When an item is found, swap it with the front element (`Move-to-Front`) or with its predecessor (`Transposition`).
  - *Advantage*: Adapts to skewed 80/20 query access distributions, achieving $O(1)$ amortized lookups for hot elements.

**3. Binary Search —**
- **Mechanical Principle**: Divide-and-conquer on random-access sorted structures.
- **How it Works**: Compares `target` against the midpoint element `arr[mid]`. Because the array is strictly monotonic:
  - If `arr[mid] == target`, match found ($O(1)$ best case).
  - If `arr[mid] < target`, the target CANNOT exist in the left half `arr[low..mid]`. Discard left half: `low = mid + 1`.
  - If `arr[mid] > target`, discard right half: `high = mid - 1`.
- **Complexity**: Halves the search space every step ($n \to n/2 \to n/4 \to \ldots \to 1$). Total steps = $\lfloor \log_2 n \rfloor + 1 \Longrightarrow \mathbf{O(\log n)}$ worst and average case. Best case $O(1)$.
- **Non-Negotiable Constraints**:
  1. Monotonicity: Data MUST be sorted.
  2. Contiguous Index Addressing: Requires $O(1)$ random memory access (`arr[mid] = base + mid * 4`). > **Crucial Exam Rule**: Binary Search CANNOT be used efficiently on linked lists! Finding the midpoint node in a linked list requires scanning $n/2$ pointers ($O(n)$), making total time $O(n)$, destroying the logarithmic speedup.

**4. Bubble Sort —**
- **Mechanical Principle**: Repeated adjacent pairwise comparisons and swaps.
- **How it Works**: Iterates through the array. In pass $i$, compares each adjacent pair (`arr[j], arr[j+1]`). If `arr[j] > arr[j+1]`, swaps them. At the end of pass $i$, the $i$-th largest element has "bubbled" to index $n - 1 - i$. The unsorted prefix shrinks by 1 each pass.
- **Optimized Early-Exit**: A boolean flag `swapped` is set to `false` at the start of each pass. If no swaps occur during an entire pass, the array is already sorted, and the algorithm breaks immediately.
- **Complexity**:
  - *Best Case*: $O(n)$ comparisons, $0$ swaps (when already sorted with `swapped` flag).
  - *Worst Case*: Array reversed. $\frac{n(n-1)}{2}$ comparisons and $\frac{n(n-1)}{2}$ swaps $\Longrightarrow \mathbf{O(n^2)}$.
  - *Average Case*: $\frac{n(n-1)}{2}$ comparisons, $\approx \frac{n^2}{4}$ swaps $\Longrightarrow \mathbf{O(n^2)}$.
- **Cost**: Each swap requires 3 memory operations (2 array writes). Worst-case writes $\approx n^2 - n$. Slowest elementary sort in practice.

**5. Selection Sort —**
- **Mechanical Principle**: Blind extremum selection and boundary placement.
- **How it Works**: Divides array into sorted prefix $arr[0..i-1]$ and unsorted suffix $arr[i..n-1]$. In iteration $i$, unconditionally scans the entire unsorted suffix to find the index `min_idx` of the global minimum. Then performs **exactly ONE swap** between `arr[i]` and `arr[min_idx]`.
- **Complexity**:
  - *Comparisons*: Completely non-adaptive! Even if the array is already sorted, it MUST compare every element in the remaining suffix to confirm no smaller element exists. Always exactly $\sum_{i=0}^{n-2}(n-1-i) = \frac{\mathbf{n(n-1)}}{\mathbf{2}} \in \mathbf{\Theta(n^2)}$ comparisons in Best, Average, and Worst cases!
  - *Swaps*: At most $n-1$ swaps across the entire sort.
- **Stability**: **UNSTABLE**. Long-distance swaps leapfrog over identical elements (e.g. `[2a, 2b, 1]` → `[1, 2b, 2a]`).
- **Ideal Use Case**: Optimal when write operations are physically expensive or destructive (e.g., flash memory, EEPROM), since it guarantees at most $n-1$ swaps.

**6. Insertion Sort —**
- **Mechanical Principle**: Incremental insertion into an expanding sorted prefix.
- **How it Works**: Divides array into sorted prefix $arr[0..i-1]$ and unsorted suffix $arr[i..n-1]$. In iteration $i$, saves `key = arr[i]`. It scans backwards from $j = i - 1$ down to 0. For every element where `arr[j] > key`, it shifts `arr[j]` one position right (`arr[j + 1] = arr[j]`). As soon as an element $\leq key$ is found (or $j < 0$), the scan stops immediately, and `key` is written into the vacated slot `arr[j + 1]`.

- **Complexity**:
  - *Best Case*: Already sorted array $\implies key \geq arr[i-1]$ on the first check. Loop terminates after 1 comparison per outer pass. Exactly $n-1$ comparisons, 0 shifts $\implies \mathbf{O(n)}$ linear time!
  - *Worst Case*: Reversed array $\implies$ every key shifts the entire prefix. $\frac{n(n-1)}{2}$ comparisons, $\frac{n(n-1)}{2}$ shifts $\implies \mathbf{O(n^2)}$.
  - *Average Case*: Random data $\implies$ key stops halfway through prefix. $\approx \frac{n^2}{4}$ comparisons, $\approx \frac{n^2}{4}$ shifts $\implies \mathbf{O(n^2)}$.
- **Stability**: **STABLE**. Strict inequality `arr[j] > key` prevents duplicate keys from crossing.
- **Shift vs Swap Advantage**: A shift requires only 1 array write ( `arr[j+1] = arr[j]` ), whereas a swap requires 2 array writes and a temporary variable. Insertion Sort does half the memory writes of Bubble Sort.

**7. Merge Sort —**

- **Mechanical Principle**: Divide-and-conquer with recursive linear merging.
- **How it Works**:
  1. *Divide*: Split array into two halves at $mid = \lfloor (low+high)/2 \rfloor$.
  2. *Conquer*: Recursively call Merge Sort on left half $arr[low..mid]$ and right half $arr[mid+1..high]$ until base case ($size \leq 1$).
  3. *Combine (Merge)*: Copy elements to an auxiliary array `aux[]` . Maintain two pointers `i = low` and `j = mid + 1` . Repeatedly take the smaller element and copy back into `arr[]` .
- **Complexity**:
  - Recurrence: $T(n) = 2T(n/2) + O(n)$.
  - Recursion tree has depth $\lceil \log_2 n \rceil$. At each level, merging takes $O(n)$ time.
  - Total Time: $\mathbf{O(n \log n)}$ **unconditionally** in Best, Average, and Worst cases!
  - *Auxiliary Space*: $\mathbf{O(n)}$ for the temporary merge buffer.
- **Stability**: **STABLE** (provided the merge step favors the left subarray on ties: `if (aux[i] <= aux[j]) arr[k] = aux[i++];` ).

**8. Quick Sort —**

- **Mechanical Principle**: Divide-and-conquer with in-place pivot partitioning.
- **How it Works**:
  1. *Partition*: Select a pivot element. Rearrange the array in-place so that all elements $\leq pivot$ are moved to the left of the pivot, and all elements $> pivot$ are moved to the right. The pivot is now at its permanent final sorted index $p$.
  2. *Recurse*: Recursively sort subarray $arr[low..p-1]$ and $arr[p+1..high]$.
- **Complexity**:
  - *Best & Average Case*: $O(n \log n)$ when pivot splits the array roughly in half.
  - *Worst Case*: $O(n^2)$ when pivot is the smallest or largest element every time (e.g. sorted input with end pivot).
  - *Space*: $O(\log n)$ call stack space for recursion. In-place data movement.
- **Stability**: **UNSTABLE** due to long-range partitioning swaps.

---

### 2.5.2 Grand Algorithmic Comparison Matrix

| Algorithm | Paradigm | Best-Case Time | Average-Case Time | Worst-Case Time | Auxiliary Space | Stable? | Adaptive? | Comparisons (Avg) | Memory Writes (Avg) |
|---|---|---|---|---|---|---|---|---|---|
| **Linear Search** | Brute Force | $O(1)$ | $O(n)$ | $O(n)$ | $O(1)$ | N/A | No | $n/2$ | 0 |
| **Sentinel Search** | Optimized Brute | $O(1)$ | $O(n)$ | $O(n)$ | $O(1)$ | N/A | No | $n/2$ (half branches) | 2 (set/restore) |
| **Binary Search** | Divide & Conquer | $O(1)$ | $O(\log n)$ | $O(\log n)$ | $O(1)$ | N/A | Yes | $\approx \log_2 n$ | 0 |
| **Bubble Sort** | Exchange | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | **YES** | Yes (flag) | $\approx n^2/2$ | $\approx 3n^2/4$ writes |
| **Selection Sort** | Selection | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | **NO** | No | Exactly $\frac{n(n-1)}{2}$ | At most $n-1$ swaps |
| **Insertion Sort** | Insertion | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ | **YES** | **YES** | $\approx n^2/4$ | $\approx n^2/4$ shifts |
| **Merge Sort** | Divide & Conquer | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ | **YES** | No | $\approx n \log_2 n$ | $2n \log_2 n$ copies |
| **Quick Sort** | Partitioning | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$ | $O(\log n)$ | **NO** | No | $\approx 1.39n \log_2 n$ | $O(n \log n)$ swaps |

---

### 2.5.3 Exam Decision Tree: Which Algorithm When?

1. **Array is small ($n \leq 30$) or nearly sorted**: $\implies$ Use **Insertion Sort** ($O(n)$ best case, tiny constant factor, zero allocation).
2. **Memory writes are physically constrained (Flash/EEPROM)**: $\implies$ Use **Selection Sort** (strictly $\leq n-1$ swaps, minimizing memory wear).
3. **Guaranteed $O(n \log n)$ runtime and stability required**: $\implies$ Use **Merge Sort** (standard in Java `Collections.sort` for objects).
4. **General-purpose fast primitive sorting with in-place memory**: $\implies$ Use **Quick Sort / Dual-Pivot QuickSort** (standard in Java `Arrays.sort` for primitives).
5. **Searching in unsorted collection with few queries**: $\implies$ Use **Linear Search** ($O(n)$, avoid $O(n \log n)$ sorting overhead).
6. **Searching in static collection with repeated queries**: $\implies$ **Sort once ($O(n \log n)$) then use Binary Search ($O(\log n)$ per query)**.

---

## SECTION 2.6: PREDICTED EXAM QUESTIONS & READY-TO-COPY ANSWERS

### 2.6.1 High-Probability Theory Questions (Ready Answer Scripts)

**Q: "Explain why Insertion Sort is faster than Bubble Sort on random data." —**

- **Answer Script**:
  1. *Comparisons*: On random data, Insertion Sort finds the correct insertion slot halfway through the sorted prefix on average, requiring $\approx \frac{n^2}{4}$ comparisons. Standard Bubble Sort always scans adjacent pairs to the end of the unsorted subarray, performing $\approx \frac{n^2}{2}$ comparisons (twice as many).
  2. *Memory Writes (Shifts vs. Swaps)*: Bubble Sort performs a complete 3-step swap per inversion (2 array writes: `tmp = a[j]; a[j] = a[j+1]; a[j+1] = tmp;` ), totaling $\approx \frac{3n^2}{4}$ memory writes. Insertion Sort shifts elements with a single write ( `a[j+1] = a[j];` ) and writes the key once, doing only $\approx \frac{n^2}{4}$ writes (one-third of Bubble Sort).
  3. *Cache Locality*: Backward sequential shifts keep accessed memory contiguous in L1 CPU cache.

**Q: "Explain why Selection Sort is O(n^2) even on an already sorted array." —**

- **Answer Script**: Selection Sort is non-adaptive. In each outer iteration $i$, it must unconditionally inspect every single element from $i+1$ to $n-1$ to verify that no smaller element exists. It cannot detect existing order. Therefore, it always executes $\sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2} \in \Theta(n^2)$ comparisons regardless of initial array ordering.

**Q: "Explain Stability in sorting algorithms and prove which are stable/unstable." —**

- **Answer Script**:
  - *Definition*: A sorting algorithm is stable if elements with equal keys appear in the output in the same relative order as they appeared in the input. Crucial in multi-key sorting (e.g. sorting genomic records by chromosome, then by start position).
  - *Insertion Sort (STABLE)*: When inserting `key` , the shift condition `a[j] > key` is strictly greater. Equal elements fail this check, causing the scan to stop. Identical keys never bypass each other.
  - *Bubble Sort (STABLE)*: Swaps only occur when `a[j] > a[j+1]` (strict inequality). Equal adjacent elements are never swapped.
  - *Selection Sort (UNSTABLE)*: Long-distance boundary swaps leapfrog identical elements. *Counterexample*: Array $[2_a, 2_b, 1]$. In iteration 1, minimum 1 swaps with $a[0]$ ($2_a$). Array becomes $[1, 2_b, 2_a]$. $2_b$ is now placed before $2_a$, inverting their input sequence.
  - *Merge Sort (STABLE)*: During merge, if `left[i] <= right[j]` , the element from the left half is chosen first, preserving initial order.
  - *Quick Sort (UNSTABLE)*: Partitioning swaps elements across the pivot over long distances.

**Q: "Why can Binary Search NOT be used efficiently on a Singly Linked List?" —**

- **Answer Script**: Binary Search requires $O(1)$ random access to jump to the midpoint $mid = (low+high)/2$. In a singly linked list, memory is non-contiguous; finding the midpoint requires traversing $n/2$ pointers ($O(n)$ time). The recurrence becomes $T(n) = T(n/2) + O(n) = O(n)$, completely destroying the $O(\log n)$ advantage and performing no better than simple linear search.

**Q: "Why is Merge Sort preferred over QuickSort for Linked Lists?" —**

- **Answer Script**:
  1. Sequential Pointer Access: Merging two sorted linked lists requires only sequential pointer rewiring in $O(1)$ extra space, whereas array Merge Sort requires $O(n)$ auxiliary buffer memory.
  2. Finding Midpoint: Fast and slow pointers find the midpoint in a single $O(n)$ pass.
  3. No Random Access Needed: QuickSort relies on random access for pivot partitioning; on linked lists, partitioning is cumbersome and exhibits poor cache performance.

---

### 2.6.2 High-Probability Data Structure Implementations

**Queue Using Two Stacks (Amortized O(1)) —**

```
public class QueueWithTwoStacks {
    private java.util.Stack<Integer> inStack = new
java.util.Stack<>();
    private java.util.Stack<Integer> outStack = new
java.util.Stack<>();

    public void enqueue(int x) { inStack.push(x); }

    public int dequeue() {
        if (isEmpty()) throw new RuntimeException("Queue
Underflow");
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty())
outStack.push(inStack.pop());
        }
        return outStack.pop();
    }

    public int front() {
        if (isEmpty()) throw new RuntimeException("Queue is
Empty");
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty())
outStack.push(inStack.pop());
        }
        return outStack.peek();
    }

    public boolean isEmpty() { return inStack.isEmpty() &&
outStack.isEmpty(); }
}
```

**Doubly Linked List Delete Node O(1) Given Pointer —**

```
// O(1) unlinking when target node reference is already given
public void deleteNode(DoublyLinkList.Node target) {
    if (target == null) return;
    if (target == first) { delete_first(); return; }
    if (target == last)  {
        last = last.previous;
        last.next = null;
        return;
    }
    // Middle node: bypass target in both directions
    target.previous.next = target.next;
    target.next.previous = target.previous;
}
```

**Singly Linked List: Delete at Index & Get K-th from End —**

```
// Delete node at 0-based index: O(n)
public void deleteAtIndex(int index) {
    if (first == null || index < 0) return;
    if (index == 0) { first = first.next; return; }
    SinglyLinkList.Node cur = first;
    for (int i = 0; cur != null && i < index - 1; i++) cur =
cur.next;
    if (cur == null || cur.next == null) return;
    cur.next = cur.next.next;
}

// Find k-th node from end (1-based): O(n) time, O(1) space
public SinglyLinkList.Node getKthFromEnd(int k) {
    if (first == null || k <= 0) return null;
    SinglyLinkList.Node fast = first, slow = first;
    for (int i = 0; i < k; i++) {
        if (fast == null) return null; // List shorter than k
        fast = fast.next;
    }
    while (fast != null) {
        fast = fast.next;
        slow = slow.next;
    }
    return slow;
}
```

---

## SECTION 2.7: ALGORITHM REVERSAL MASTER BLUEPRINT (ASCENDING <-> DESCENDING)

When an exam problem asks to sort, search, or maintain a data structure in **descending order** (or flip Min ↔ Max), **DO NOT rewrite the algorithm from scratch**. Modify only the specific comparison operator or pointer branch:

**Master Comparison Reversal Table —**

| Algorithm / Template | Standard (Ascending) Line | Reversal (Descending) Modification | Mechanical Effect |
|---|---|---|---|
| **Binary Search** | `if (a[mid] < target) low = mid + 1;` | `if (a[mid] < target) high = mid - 1;` | In descending array, larger elements are on the left. |
| **Bubble Sort** | `if (a[j] > a[j + 1]) swap(a, j, j + 1);` | `if (a[j] < a[j + 1]) swap(a, j, j + 1);` | Smallest elements bubble to the end; largest stay at front. |
| **Selection Sort (Min-Front)** | `if (a[j] < a[minIdx]) minIdx = j;` | `if (a[j] > a[maxIdx]) maxIdx = j;` | Finds global MAXIMUM; swaps maximum to front index `i` . |
| **Selection Sort (Max-End)** | `if (a[j] > a[maxIdx]) maxIdx = j;` | `if (a[j] < a[minIdx]) minIdx = j;` | Finds global MINIMUM; swaps minimum to end index `end` . |
| **Insertion Sort** | `while (j >= 0 && a[j] > key)` | `while (j >= 0 && a[j] < key)` | Shifts elements SMALLER than key right; largest keys placed at index 0. |
| **Merge Sort** | `if (aux[j] < aux[i]) a[k] = aux[j++];` | `if (aux[j] > aux[i]) a[k] = aux[j++];` | Pulls larger element first. Stable ties: `if (aux[i] >= aux[j])` |
| **Quick Sort (Partition)** | `if (a[j] <= pivot) { i++; swap(); }` | `if (a[j] >= pivot) { i++; swap(); }` | Moves elements GREATER than pivot to the left partition. |
| **Priority Queue (Java)** | `new PriorityQueue<>()` (Min-Heap) | `new PriorityQueue<>(Collections.reverseOrder())` | Extracts maximum element first instead of minimum. |
| **Priority Queue (Heapify)** | `if (arr[child] < arr[parent])` | `if (arr[child] > arr[parent])` | Inverts heap invariant from Min-Heap ( `P <= C` ) to Max-Heap ( `P >= C` ). |
| **Sorted List Insert** | `while (cur.next != null && cur.next.key < val)` | `while (cur.next != null && cur.next.key > val)` | Scans past elements LARGER than `val` , keeping descending order. |
| **Monotonic Stack (NGE/NSE)** | `while (!stack.isEmpty() && a[peek] <= a[i])` | `while (!stack.isEmpty() && a[peek] >= a[i])` | Changes from Next Greater Element to Next Smaller Element. |
| **Stack Sort (1 Aux Stack)** | `while (!aux.isEmpty() && aux.peek() > tmp)` | `while (!aux.isEmpty() && aux.peek() < tmp)` | Yields descending order (largest on top) instead of ascending. |

**The Two Golden Rules of Algorithm Reversal —**

1. **Sorting / Priority / Scanning**: Invert the inequality operator between candidate elements ( `<` becomes `>` , `>` becomes `<` ).
2. **Binary Search on Inverted Arrays**: Because elements decrease as indices increase, if `a[mid] < target` , the target is *larger*, meaning it must be located at a *lower index* on the left $\implies$ set `high = mid - 1` (opposite of standard binary search!).

---

## SECTION 3: ARRAYS, TWO-POINTERS & SLIDING WINDOWS

### 3.1 Prefix Sum Array (Range Query in O(1))

Used to compute subarray sum `arr[L..R]` in $O(1)$ time after $O(n)$ precomputation.

```
public class PrefixSum {
    // 1-based indexing prefix array: pref[i] = sum(arr[0..i-1])
    public static int[] buildPrefix(int[] arr) {
        int n = arr.length;
        int[] pref = new int[n + 1];
        for (int i = 0; i < n; i++) {
            pref[i + 1] = pref[i] + arr[i];
        }
        return pref;
    }

    // Sum of elements from index L to R inclusive: O(1)
    public static int queryRange(int[] pref, int L, int R) {
        return pref[R + 1] - pref[L];
    }
}
```

### 3.2 Difference Array (Range Updates in O(1))

Allows adding `val` to all elements in range `[L, R]` in $O(1)$ time.

```
public class DifferenceArray {
    private int[] diff;

    public DifferenceArray(int[] arr) {
        int n = arr.length;
        diff = new int[n];
        diff[0] = arr[0];
        for (int i = 1; i < n; i++) diff[i] = arr[i] - arr[i - 1];
    }

    // Add val to range [L, R] in O(1)
    public void update(int L, int R, int val) {
        diff[L] += val;
        if (R + 1 < diff.length) diff[R + 1] -= val;
    }

    // Reconstruct final array in O(n)
    public int[] getResult() {
        int[] res = new int[diff.length];
        res[0] = diff[0];
        for (int i = 1; i < diff.length; i++) res[i] = res[i - 1] +
diff[i];
        return res;
    }
}
```

### 3.3 Two-Pointers: Converging (Opposite Ends)

```
// Reverse array in-place: O(n) time, O(1) space
public static void reverseArray(int[] a) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        int tmp = a[left];
        a[left] = a[right];
        a[right] = tmp;
        left++;
        right--;
    }
}

// Two Sum in Sorted Array: find indices where a[i] + a[j] ==
target
public static boolean twoSumSorted(int[] a, int target) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        int sum = a[left] + a[right];
        if (sum == target) return true;
        else if (sum < target) left++;
        else right--;
    }
    return false;
}
```

### 3.4 Sliding Window: Fixed Size K

```
// Maximum sum of any contiguous subarray of fixed size K: O(n)
time
public static int maxSubarraySumFixed(int[] a, int k) {
    if (a.length < k) return -1;
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += a[i];
    int maxSum = windowSum;

    for (int i = k; i < a.length; i++) {
        windowSum += a[i] - a[i - k]; // Slide: add incoming,
remove outgoing
        if (windowSum > maxSum) maxSum = windowSum;
```

```
    }
    return maxSum;
}
```

### 3.5 Sliding Window: Variable Size

```
// Smallest subarray with sum >= target: O(n) time
public static int minSubArrayLen(int target, int[] nums) {
    int left = 0, currentSum = 0, minLen = Integer.MAX_VALUE;
    for (int right = 0; right < nums.length; right++) {
        currentSum += nums[right];
        while (currentSum >= target) {
            minLen = Math.min(minLen, right - left + 1);
            currentSum -= nums[left];
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
```

## SECTION 4: SEARCHING ALGORITHMS

### 4.1 Linear Search (Standard & Sentinel)

```
// Standard Linear Search: O(n) worst, O(1) best
public static int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

### 4.2 Standard Binary Search (Iterative & Recursive)

**Invariant**: Array MUST be sorted in ascending order.

```
// Iterative Binary Search: O(log n) time, O(1) space
public static int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents 32-bit integer overflow
        if (a[mid] == target) return mid;
        else if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // Not found
}

// Recursive Binary Search: O(log n) time, O(log n) call stack space
public static int binarySearchRec(int[] a, int low, int high, int target) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (a[mid] == target) return mid;
    if (a[mid] < target) return binarySearchRec(a, mid + 1, high, target);
    return binarySearchRec(a, low, mid - 1, target);
}
```

### 4.3 Binary Search in Descending Array (Template Modification)

In a descending array, larger elements are at lower indices. In the standard `binarySearch` template above, change only the search branches: `if (a[mid] < target) high = mid - 1; else low = mid + 1;` *(Reverses pointer advance direction)*.

### 4.4 Binary Search: First Occurrence (Lower Bound) & Last Occurrence

```
// First occurrence (Lower Bound): index of first element >= target
public static int findFirstOccurrence(int[] a, int target) {
    int low = 0, high = a.length - 1, result = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) {
            result = mid;
            high = mid - 1; // Keep searching left to find earlier match!
        } else if (a[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return result;
}

// Last occurrence (Upper Bound): index of last element <= target
public static int findLastOccurrence(int[] a, int target) {
    int low = 0, high = a.length - 1, result = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) {
            result = mid;
            low = mid + 1; // Keep searching right to find later match!
        } else if (a[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return result;
}
```

## SECTION 5: ELEMENTARY SORTING ALGORITHMS

### 5.1 Bubble Sort (With Early Exit & Counter Instrumentation)

Bubbles the largest unsorted element to the end of the array each pass.

```
public class BubbleSort {
    public static void sort(int[] a) {
        int n = a.length;
        boolean swapped;
        // Pass i: largest elements bubble to index (n - 1 - i)
        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            for (int j = 0; j < n - 1 - i; j++) {
                if (a[j] > a[j + 1]) { // Strictly > preserves STABILITY
                    int tmp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = tmp;
                    swapped = true;
                }
            }
            // If no swaps occurred in an entire pass, array is already sorted!
            if (!swapped) break; // Gives O(n) best-case time
        }
    }
}
```

### 5.2 Selection Sort: Standard Min-to-Front (Ascending)

Scans unsorted suffix to find minimum, swaps into boundary position $i$.

```
public class SelectionSort {
    public static void sortMinToFront(int[] a) {
        int n = a.length;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (a[j] < a[minIdx]) {
                    minIdx = j;
                }
            }
            // Swap minimum into index i
            if (minIdx != i) {
                int tmp = a[i];
                a[i] = a[minIdx];
                a[minIdx] = tmp;
            }
        }
    }
}
```

### 5.3 Selection Sort Variant: Max-to-End (Template Modification)

To swap maximum elements to the end instead of minimums to the front (Mock Exam 2): 1. **Outer loop backward**: `for (int end = n - 1; end > 0; end--)`. 2. **Find maximum in prefix**: scan `j = 1` to `end`, if `a[j] > a[maxIdx]` set `maxIdx = j`. 3. **Swap with end**: `int tmp = a[end]; a[end] = a[maxIdx]; a[maxIdx] = tmp;`.

### 5.4 Insertion Sort: Standard Ascending (With Shifts)

Divides array into sorted prefix $a[0..i-1]$ and unsorted suffix. Takes `key = a[i]`, shifts larger elements right, drops `key` into slot.

```
public class InsertionSort {
    public static void sortAscending(int[] a) {
        int n = a.length;
        for (int i = 1; i < n; i++) {
            int key = a[i];
            int j = i - 1;
            // Shift elements of sorted prefix that are > key to one position right
            while (j >= 0 && a[j] > key) {
                a[j + 1] = a[j]; // 1 memory write per shift (no 3-step swap!)
                j--;
            }
            a[j + 1] = key; // Place key into its hole
        }
    }

    // Descending Modification (Mock Exam 1):
    // In while-condition, change `a[j] > key` to `a[j] < key` (shifts smaller elements right).
            a[j + 1] = key;
        }
    }
}
```

### 5.5 Stability Proofs & Analysis

- **Definition**: A sorting algorithm is **stable** if elements with identical keys maintain their relative input order in the output.
- **Insertion Sort is STABLE**: The shift condition `a[j] > key` is strict. When `a[j] == key`, the condition is false; the loop terminates immediately, and `key` is placed after `a[j]`. They never cross.
- **Selection Sort is UNSTABLE**: Long-range swaps can jump an element across an identical key.
  - *Counterexample*: `[2_a, 2_b, 1]`. Min is `1` at index 2. Swap `a[0]` ($2_a$) with `a[2]` (1). Array becomes `[1, 2_b, 2_a]`. Relative order of $2_a$ and $2_b$ is inverted!
- **Best-Case Comparisons**:
  - Insertion Sort: Already sorted $\Longrightarrow a[j] > key$ fails on first check. Exactly $n-1$ comparisons, 0 shifts $\Longrightarrow \mathbf{O(n)}$.
  - Selection Sort: Non-adaptive $\Longrightarrow$ outer loop runs $n-1$ times; inner loop scans to end. Always $\frac{n(n-1)}{2} \in \mathbf{O(n^2)}$ comparisons.

## SECTION 6: STACKS & STACK APPLICATIONS

### 6.1 Array-Based Stack (ArrayStack)

```
public class ArrayStack {
    private int[] data;
    private int top; // Index of current top element (-1 when empty)
    private int capacity;

    public ArrayStack(int cap) {
        this.capacity = cap;
        this.data = new int[cap];
        this.top = -1;
    }

    public boolean isEmpty() { return top == -1; }
    public boolean isFull()  { return top == capacity - 1; }
    public int size()        { return top + 1; }

    public void push(int x) {
        if (isFull()) throw new RuntimeException("Stack Overflow");
        data[++top] = x; // Increment top first, then assign
    }

    public int pop() {
        if (isEmpty()) throw new RuntimeException("Stack Underflow");
        return data[top--]; // Return element, then decrement top
    }

    public int peek() {
        if (isEmpty()) throw new RuntimeException("Stack is Empty");
        return data[top];
    }
}
```

### 6.2 Linked List-Based Stack (LinkedStack)

Nodes are pushed and popped at the head of a singly linked list in $O(1)$ time.

```
public class LinkedStack {
    private static class Node {
        int key;
        Node next;
        Node(int k, Node n) { this.key = k; this.next = n; }
    }
    private Node first = null;
    private int size = 0;

    public boolean isEmpty() { return first == null; }
    public int size()        { return size; }

    public void push(int x) {
        first = new Node(x, first); // New node becomes new head
        size++;
    }

    public int pop() {
        if (isEmpty()) throw new RuntimeException("Stack Underflow");
        int val = first.key;
        first = first.next; // Advance head
        size--;
        return val;
    }

    public int peek() {
        if (isEmpty()) throw new RuntimeException("Stack is Empty");
        return first.key;
    }
}
```

### 6.3 Balanced Parentheses / Brackets Matching Template

```
public static boolean isBalanced(String s) {
    java.util.Stack<Character> stack = new java.util.Stack<>();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == ']' && top != '[') return false;
            if (c == '}' && top != '{') return false;
        }
    }
    return stack.isEmpty(); // If stack has unmatched opening brackets, false
}
```

### 6.4 Monotonic Stack: Next Greater Element (NGE)

Finds the first element to the right that is strictly greater than `a[i]`.

```
public static int[] nextGreaterElement(int[] a) {
    int n = a.length;
    int[] result = new int[n];
    java.util.Stack<Integer> stack = new java.util.Stack<>(); // Stores INDICES

    // Traverse from right to left
    for (int i = n - 1; i >= 0; i--) {
        // Pop elements smaller than or equal to current
        while (!stack.isEmpty() && a[stack.peek()] <= a[i]) {
            stack.pop();
        }
        result[i] = stack.isEmpty() ? -1 : a[stack.peek()];
        stack.push(i);
    }
    return result;
}
```

### 6.5 Stack Sorting Using Strictly ONE Auxiliary Stack (Mock Exam 2 Q5)

```
public static void sortStack(java.util.Stack<Integer> input) {
    if (input == null || input.isEmpty()) return;
    java.util.Stack<Integer> aux = new java.util.Stack<>();

    while (!input.isEmpty()) {
        int tmp = input.pop();
        // Shift elements in aux that are greater than tmp back into input
        while (!aux.isEmpty() && aux.peek() > tmp) {
            input.push(aux.pop());
        }
        aux.push(tmp);
    }

    // aux has largest on top; pouring back to input puts SMALLEST ON TOP!
    while (!aux.isEmpty()) {
        input.push(aux.pop());
    }
}
```

## SECTION 7: QUEUES, DEQUES & PRIORITY QUEUES

### 7.1 Circular Array Queue (CircularQueue / Ring Buffer)

Avoids shifting elements by wrapping `front` and `rear` pointers using modulo $N$.

```
public class CircularQueue {
    private int[] data;
    private int front; // Points to front element
    private int rear;  // Points to rear element
    private int size;
    private int capacity;

    public CircularQueue(int cap) {
        this.capacity = cap;
        this.data = new int[cap];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    public boolean isEmpty() { return size == 0; }
    public boolean isFull()  { return size == capacity; }
    public int size()        { return size; }

    public void enqueue(int x) {
        if (isFull()) throw new RuntimeException("Queue Overflow");
        rear = (rear + 1) % capacity; // Circular wrap
        data[rear] = x;
        size++;
    }

    public int dequeue() {
        if (isEmpty()) throw new RuntimeException("Queue Underflow");
        int val = data[front];
        front = (front + 1) % capacity; // Circular wrap
        size--;
        return val;
    }

    public int front() {
        if (isEmpty()) throw new RuntimeException("Queue is Empty");
        return data[front];
    }
}
```

### 7.2 Linked List-Based Queue (LinkedQueue)

Maintains `first` (head) for $O(1)$ dequeue and `last` (tail) for $O(1)$ enqueue.

```
public class LinkedQueue {
    private static class Node {
```

```
        int key;
        Node next;
        Node(int k) { this.key = k; this.next = null; }
    }
    private Node first = null;
    private Node last = null;
    private int size = 0;

    public boolean isEmpty() { return first == null; }
    public int size()        { return size; }

    public void enqueue(int x) {
        Node newNode = new Node(x);
        if (isEmpty()) {
            first = newNode;
            last = newNode;
        } else {
            last.next = newNode;
            last = newNode;
        }
        size++;
    }

    public int dequeue() {
        if (isEmpty()) throw new RuntimeException("Queue
Underflow");
        int val = first.key;
        first = first.next;
        if (first == null) last = null; // List became empty
        size--;
        return val;
    }
}
```

### 7.3 Array-Based Deque (Double-Ended Queue with 4 O(1) Operations)

Required in 2023 Midterm Exam Question 5.

```
public class ArrayDeque {
    private int[] arr;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public ArrayDeque(int cap) {
        this.capacity = cap;
        this.arr = new int[cap];
        this.front = -1;
        this.rear = -1;
        this.size = 0;
    }

    public boolean isFull()  { return size == capacity; }
    public boolean isEmpty() { return size == 0; }

    // 1. Insert at Front: front moves counter-clockwise
    public void insertFront(int x) {
        if (isFull()) throw new RuntimeException("Deque
Overflow");
        if (isEmpty()) {
            front = 0; rear = 0;
        } else {
            front = (front - 1 + capacity) % capacity; // Modular
backward step
        }
        arr[front] = x;
        size++;
    }

    // 2. Insert at Rear: rear moves clockwise
    public void insertRear(int x) {
        if (isFull()) throw new RuntimeException("Deque
Overflow");
        if (isEmpty()) {
            front = 0; rear = 0;
        } else {
            rear = (rear + 1) % capacity; // Modular forward step
        }
        arr[rear] = x;
        size++;
    }

    // 3. Delete from Front: front moves clockwise
    public int deleteFront() {
        if (isEmpty()) throw new RuntimeException("Deque
Underflow");
        int val = arr[front];
        if (front == rear) { // Reset to empty when last element
removed
            front = -1; rear = -1;
        } else {
            front = (front + 1) % capacity;
        }
        size--;
        return val;
    }

    // 4. Delete from Rear: rear moves counter-clockwise
    public int deleteRear() {
        if (isEmpty()) throw new RuntimeException("Deque
Underflow");
        int val = arr[rear];
        if (front == rear) { // Reset to empty
            front = -1; rear = -1;
        } else {
            rear = (rear - 1 + capacity) % capacity;
        }
        size--;
        return val;
    }
}
```

### 7.4 Priority Queue & Binary Heap Indexing

Binary heaps are represented in a contiguous array without explicit pointers: - Root is at index `0`. - Parent of node $i$: `parent(i) = (i - 1) / 2`. - Left Child of node $i$: `leftChild(i) = 2 * i + 1`. - Right Child of node $i$: `rightChild(i) = 2 * i + 2`. - **Max Heap Invariant**: `arr[parent] >= arr[child]`. `dequeue()` extracts maximum. - **Min Heap Invariant**: `arr[parent] <= arr[child]`. `dequeue()` extracts minimum.

### 7.5 Queue Reversal of First K Elements (Mock Exam 1 Q5)

```
public static void reverseFirstK(java.util.Queue<Integer> q, int
k) {
    if (q == null || k <= 0 || k > q.size()) return;
    java.util.Stack<Integer> stack = new java.util.Stack<>();

    // Step 1: Dequeue first k elements and push to stack (LIFO
order)
    for (int i = 0; i < k; i++) {
        stack.push(q.poll());
    }

    // Step 2: Enqueue stack items back into queue (now reversed
at back)
    while (!stack.isEmpty()) {
        q.add(stack.pop());
    }

    // Step 3: Rotate remaining (size - k) elements to preserve
original order
    int remaining = q.size() - k;
    for (int i = 0; i < remaining; i++) {
        q.add(q.poll());
    }
}
```

## SECTION 8: LINKED LISTS IN-DEPTH

### 8.1 Singly Linked List Implementation Template

```
public class SinglyLinkList {
    public static class Node {
        public int key;
        public Node next;
        public Node(int k) { this.key = k; this.next = null; }
    }
    public Node first = null;

    // Insert at front: O(1)
    public void insertFirst(int val) {
        Node n = new Node(val);
        n.next = first;
        first = n;
    }

    // Insert into sorted (ascending) list: O(n) (Mock Exam 1 Q3a)
    public void insertNode(int value) {
        Node newNode = new Node(value);
        if (first == null || value <= first.key) {
            newNode.next = first;
            first = newNode;
            return;
        }
        Node cur = first;
        while (cur.next != null && cur.next.key < value) {
            cur = cur.next;
        }
        newNode.next = cur.next;
        cur.next = newNode;
    }

    // Delete all nodes with given value: O(n) (Mock Exam 1 Q3b)
    public void deleteAllNodes(int value) {
        while (first != null && first.key == value) {
            first = first.next;
        }
        if (first == null) return;
        Node cur = first;
        while (cur.next != null) {
            if (cur.next.key == value) {
                cur.next = cur.next.next; // Bypass matching node
            } else if (cur.next.key > value) {
                break; // Early exit optimization if list is
sorted!
            } else {
                cur = cur.next;
            }
        }
    }

    // Move last node to front: O(n) (Mock Exam 2 Q3a)
    public void moveLastToFront() {
        if (first == null || first.next == null) return;
        Node secLast = first;
        while (secLast.next.next != null) {
            secLast = secLast.next;
        }
        Node last = secLast.next;
        secLast.next = null; // Unlink last
        last.next = first;   // Prepend to front
        first = last;        // Update head
    }

    // Remove duplicates from sorted list: O(n) (Mock Exam 2 Q3b)
    public void removeDuplicates() {
        if (first == null) return;
        Node cur = first;
        while (cur.next != null) {
            if (cur.key == cur.next.key) {
                cur.next = cur.next.next; // Bypass duplicate, DO
NOT advance cur
            } else {
                cur = cur.next;
            }
        }
    }

    // Reverse list iteratively (3-pointer pattern): O(n) time,
O(1) space
    public void reverse() {
        Node prev = null;
        Node cur = first;
        Node next = null;
        while (cur != null) {
            next = cur.next; // Save next node
            cur.next = prev; // Reverse arrow
            prev = cur;      // Advance prev
            cur = next;      // Advance cur
        }
        first = prev;        // Update head
    }

    // Find Middle Node (Slow & Fast Pointer / Tortoise & Hare):
O(n)
    public Node findMiddle() {
        if (first == null) return null;
        Node slow = first;
        Node fast = first;
        while (fast != null && fast.next != null) {
            slow = slow.next;       // 1 step
            fast = fast.next.next;  // 2 steps
        }
        return slow; // Points to middle (or second middle if
even)
    }

    // Detect Cycle (Floyd's Cycle-Finding Algorithm)
    public boolean hasCycle() {
        Node slow = first, fast = first;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true; // Collision indicates
cycle
        }
        return false;
    }
}
```

### 8.2 Doubly Linked List Implementation Template

```
public class DoublyLinkList {
    public static class Node {
        public int key;
        public Node next;
        public Node previous;
        public Node(int k) { this.key = k; }
    }
    public Node first = null;
    public Node last = null;

    // Delete first node: O(1) (Real Exam 2023 Q3a)
    public void delete_first() {
        if (first == null) return;
        if (first == last) { // Single node
            first = null;
            last = null;
        } else {
            first = first.next;
            first.previous = null;
        }
    }

    // Insert at last: O(1) (Real Exam 2023 Q3b)
    public void insert_last(int key) {
        Node newNode = new Node(key);
        if (first == null) {
            first = newNode;
            last = newNode;
        } else {
            last.next = newNode;
            newNode.previous = last;
            last = newNode;
        }
    }

    // Merge two sorted doubly linked lists into a new sorted
list: O(n1 + n2) (Real Exam 2023 Q3c)
    public static DoublyLinkList merge(DoublyLinkList L1,
DoublyLinkList L2) {
        DoublyLinkList result = new DoublyLinkList();
        if (L1 == null && L2 == null) return result;
        if (L1 == null || L1.first == null) return L2;
        if (L2 == null || L2.first == null) return L1;

        Node p1 = L1.first;
        Node p2 = L2.first;

        while (p1 != null && p2 != null) {
            if (p1.key <= p2.key) {
                result.insert_last(p1.key);
                p1 = p1.next;
            } else {
                result.insert_last(p2.key);
                p2 = p2.next;
            }
        }
        while (p1 != null) {
            result.insert_last(p1.key);
            p1 = p1.next;
        }
        while (p2 != null) {
            result.insert_last(p2.key);
            p2 = p2.next;
        }
        return result;
    }
}
```

### 8.3 The 4x5 Linked List Complexity Matrix (Real Exam 2023 Q4)

| Operation | Unsorted, Singly List | Sorted Singly List | Unsorted, Doubly List | Sorted Doubly List |
|---|---|---|---|---|
| **Search** | $O(n)$ | $O(n)$ | $O(n)$ | $O(n)$ |
| **Insert a new value** | $O(1)$ at head | $O(n)$ search to slot | $O(1)$ at head | $O(n)$ search to slot |
| **Delete a given value** | $O(n)$ search | $O(n)$ search | $O(n)$ search | $O(n)$ search |
| **Get minimum** | $O(n)$ scan | $O(1)$ head | $O(n)$ scan | $O(1)$ head |
| **Get maximum** | $O(n)$ scan | $O(n)$ ($O(1)$ w/ tail) | $O(n)$ scan | $O(1)$ tail |

**EXAM TRAP EXPLANATIONS**: 1. **Sorted List Insert is NOT $O(1)$**: Splicing takes $O(1)$, but locating the sorted position requires scanning $O(n)$ nodes! Total: $O(n)$. 2. **Delete by Value is NOT $O(1)$**: Even in a doubly linked list where node removal is $O(1)$, you must first *search* for the node containing that value ($O(n)$). Total: $O(n)$.

## SECTION 9: RECURSION & MERGE SORT

### 9.1 Recursion Golden Rules & Mental Models

1. **Base Case**: Must have at least one branch that returns without recurring (prevents infinite recursion / `StackOverflowError`).
2. **Progress**: Every recursive call must move strictly closer to the base case (e.g., $n-1$ or $n/2$).
3. **Call Stack**: Each recursive invocation pushes a stack frame with its own local variables. Space complexity = maximum depth of recursion tree.

### 9.2 Merge Sort for Arrays (Full Implementation)

```
public class MergeSort {
    public static void sort(int[] a) {
        if (a == null || a.length <= 1) return;
        int[] aux = new int[a.length];
        mergeSort(a, aux, 0, a.length - 1);
    }

    private static void mergeSort(int[] a, int[] aux, int low, int
high) {
        if (low >= high) return; // Base case: 1 element
        int mid = low + (high - low) / 2;
        mergeSort(a, aux, low, mid);      // Sort left half
        mergeSort(a, aux, mid + 1, high); // Sort right half
        merge(a, aux, low, mid, high);    // Merge two sorted
halves
    }

    private static void merge(int[] a, int[] aux, int low, int
mid, int high) {
        // Copy to auxiliary array
        for (int k = low; k <= high; k++) aux[k] = a[k];

        int i = low;      // Pointer for left half
        int j = mid + 1;  // Pointer for right half

        // Merge back into a[low..high]
        for (int k = low; k <= high; k++) {
            if (i > mid)              a[k] = aux[j++];
            else if (j > high)        a[k] = aux[i++];
            else if (aux[j] < aux[i]) a[k] = aux[j++];
            else                      a[k] = aux[i++]; // Stable:
picks left if equal!
        }
    }
}
```

## SECTION 10: FULL MOCK EXAM 1 WALKTHROUGH & CODE

## Question 1: Sorting Efficiency & Runtime (20 Marks)

- **Insertion Sort**: Adaptive. Takes `key = a[i]`, shifts elements $> key$ backwards by 1 slot. On random data, stops halfway: $\approx \frac{n^2}{4}$ comparisons, $\approx \frac{n^2}{4}$ single shifts (fastest in cache).
- **Bubble Sort**: Non-adaptive pairwise swaps. Scans adjacent pairs pass-by-pass: $\approx \frac{n^2}{2}$ comparisons, $\approx \frac{3n^2}{4}$ memory writes from 3-step swaps (slowest).
- **Selection Sort**: Non-adaptive global scan. Unconditionally scans remaining unsorted subarray for minimum: always $\frac{n(n-1)}{2}$ comparisons. Fewest swaps ($\leq n-1$), but comparisons dominate in RAM. *(See Section 2.6.1 for full model answer script).*

## Question 2: Sorting (15 Marks)

**Problem Statement —** Given the sequence $S$:

$$17, 72, 99, 32, 58, 70, 44, 12, 23$$

Provide the detailed steps to arrange $S$ in **descending order** using the Insertion Sort algorithm, showing the changes in $S$ at each iteration.

**Step-by-Step Execution —**

- **Input Array Size**: $n = 9$
- **Target Order**: Descending (largest to smallest, so $A[j] < \text{key}$ causes elements to shift right).
- **Initial State**:

$$[17, 72, 99, 32, 58, 70, 44, 12, 23]$$

Sorted prefix initially contains index `0`: `[17]`.

| Iteration | Key | Comparisons & Shifts | Array State After Iteration |
|---|---|---|---|
| **Initial** | — | Unsorted input | `[17, 72, 99, 32, 58, 70, 44, 12, 23]` |
| $i = 1$ | **72** | $17 < 72 \Longrightarrow$ shift 17 to right; insert 72 at index 0 | `[72, 17, 99, 32, 58, 70, 44, 12, 23]` |
| $i = 2$ | **99** | $17 < 99 \Longrightarrow$ shift 17; $72 < 99 \Longrightarrow$ shift 72; insert 99 at index 0 | `[99, 72, 17, 32, 58, 70, 44, 12, 23]` |
| $i = 3$ | **32** | $17 < 32 \Longrightarrow$ shift 17; $72 > 32 \Longrightarrow$ stop; insert 32 at index 2 | `[99, 72, 32, 17, 58, 70, 44, 12, 23]` |
| $i = 4$ | **58** | $17 < 58 \Longrightarrow$ shift 17; $32 < 58 \Longrightarrow$ shift 32; $72 > 58 \Longrightarrow$ stop; insert 58 at index 2 | `[99, 72, 58, 32, 17, 70, 44, 12, 23]` |
| $i = 5$ | **70** | Shift 17, 32, 58; $72 > 70 \Longrightarrow$ stop; insert 70 at index 2 | `[99, 72, 70, 58, 32, 17, 44, 12, 23]` |
| $i = 6$ | **44** | Shift 17, 32, 58 $> 44 \Longrightarrow$ stop; insert 44 at index 4 | `[99, 72, 70, 58, 44, 32, 17, 12, 23]` |
| $i = 7$ | **12** | $17 > 12 \Longrightarrow$ stop immediately (0 shifts); insert 12 at index 7 | `[99, 72, 70, 58, 44, 32, 17, 12, 23]` |
| $i = 8$ | **23** | Shift 12, 17; $32 > 23 \Longrightarrow$ stop; insert 23 at index 6 | `[99, 72, 70, 58, 44, 32, 23, 17, 12]` |

**Final Result —**

$$[\mathbf{99}, \mathbf{72}, \mathbf{70}, \mathbf{58}, \mathbf{44}, \mathbf{32}, \mathbf{23}, \mathbf{17}, \mathbf{12}]$$

## Question 3: Linked List (35 Marks)

**Problem Statement —** Given the sorted (ascending order) `SinglyLinkList` class below:

```
public class SinglyLinkList {
    private Node first;

    public void insertNode(int value) {
        // YOUR CODE
    }

    public void deleteAllNodes(int value) {
        // YOUR CODE
    }
}

public class Node {
    private int key;
    private Node next;
}
```

- **a)** (15 marks) Complete the `insertNode` method that inserts a `Node` with a given integer into the sorted list at the right location.
- **b)** (20 marks) Complete the `deleteAllNodes` method that deletes all occurrences of a given integer value from the list.

**Solution 3(a): `insertNode(int value)` —**

**Algorithm & Edge Case Walkthrough**

Because the list is maintained in **ascending order**: 1. **Case 1 (Empty list or Insert at Front)**: If `first == null` or the new `value <= first.key`, the new node becomes the new head of the list. Its `next` points to the old `first`, and `first` is updated. 2. **Case 2 (Insert Middle or End)**: Traverse the list using a pointer `current` starting at `first`. We advance while `current.next != null` and `current.next.key < value`. Once the loop terminates, `current` points to the exact node after which the new node must be spliced:

`java  newNode.next = current.next;  current.next = newNode;`

**Java Implementation**

```
public void insertNode(int value) {
    Node newNode = new Node();
    newNode.key = value;
    newNode.next = null;

    // Edge Case: List is empty OR value belongs before the first node
    if (first == null || value <= first.key) {
        newNode.next = first;
        first = newNode;
        return;
    }

    // Traverse to locate the node immediately preceding the insertion point
    Node current = first;
    while (current.next != null && current.next.key < value) {
        current = current.next;
    }

    // Splice the new node into the list
    newNode.next = current.next;
    current.next = newNode;
}
```

- **Time Complexity**: $\mathcal{O}(n)$ in the worst case (inserting at the end of the list), $\mathcal{O}(1)$ in the best case (inserting before the head).
- **Space Complexity**: $\mathcal{O}(1)$ auxiliary space.

**Solution 3(b): `deleteAllNodes(int value)` —**

**Algorithm & Edge Case Walkthrough**

Deletions in a singly linked list require updating the `next` reference of the *preceding* node: 1. **Head Cleanup**: The target `value` might appear at the head of the list (one or multiple times, e.g., `20 -> 20 -> 30`). We must advance `first = first.next` until `first == null` or `first.key != value`. 2. **Body Cleanup**: After trimming the head, if the list is empty, we stop. Otherwise, we traverse using `current = first`. - If `current.next.key == value`, we bypass that node: `current.next = current.next.next;` (we do *not* advance `current` yet, because the newly linked node might also contain `value`). - Because the list is sorted in ascending order, if `current.next.key > value`, no further occurrences of `value` can exist downstream, so we can terminate early! - Otherwise, advance `current = current.next`.

**Java Implementation**

```
public void deleteAllNodes(int value) {
    // Step 1: Remove all matching occurrences at the head of the list
    while (first != null && first.key == value) {
        first = first.next;
    }

    // If the list is now empty, nothing left to delete
    if (first == null) {
        return;
    }

    // Step 2: Remove matching occurrences in the remainder of the list
    Node current = first;
    while (current.next != null) {
        if (current.next.key == value) {
            // Bypass the target node (unlinks it from the chain)
            current.next = current.next.next;
        } else if (current.next.key > value) {
            // Optimization: since the list is sorted, we can terminate early
            break;
        } else {
            current = current.next;
        }
    }
}
```

- **Time Complexity**: $\mathcal{O}(n)$ single pass over the list.
- **Space Complexity**: $\mathcal{O}(1)$ auxiliary memory.

## Question 4: Queue and Stack (20 Marks)

**4(a) Max Priority Queue Operations (10 Marks) —**

**Problem Statement**

Show the queue after each operation of the following sequence of **MAX PRIORITY QUEUE** operations, if executed on an initially empty queue:

```
enqueue(1), enqueue(20), dequeue(), enqueue(30), enqueue(4), dequeue(), dequeue(),
enqueue(50), enqueue(0), dequeue(), enqueue(2), enqueue(6), dequeue(), dequeue(),
enqueue(100), dequeue(), dequeue()
```

**Behavioral Rules of Max Priority Queue**

- `enqueue(x)`: Adds item $x$ into the priority queue.
- `dequeue()`: Extracts and returns the item with the **maximum numerical value**.
- Mental Model: Like an automated variant caller queue that always pulls the genetic variant with the highest quality/confidence score first.

**Step-by-Step Trace**

| Step | Operation | Action & Priority Queue State (Ordered Highest to Lowest) | Value Returned |
|---|---|---|---|
| 1 | `enqueue(1)` | `[1]` | — |
| 2 | `enqueue(20)` | `[20, 1]` | — |
| 3 | `dequeue()` | Remove max element (20) $\Longrightarrow$ Queue: `[1]` | **20** |
| 4 | `enqueue(30)` | `[30, 1]` | — |
| 5 | `enqueue(4)` | `[30, 4, 1]` | — |
| 6 | `dequeue()` | Remove max element (30) $\Longrightarrow$ Queue: `[4, 1]` | **30** |
| 7 | `dequeue()` | Remove max element (4) $\Longrightarrow$ Queue: `[1]` | **4** |
| 8 | `enqueue(50)` | `[50, 1]` | — |
| 9 | `enqueue(0)` | `[50, 1, 0]` | — |
| 10 | `dequeue()` | Remove max element (50) $\Longrightarrow$ Queue: `[1, 0]` | **50** |
| 11 | `enqueue(2)` | `[2, 1, 0]` | — |
| 12 | `enqueue(6)` | `[6, 2, 1, 0]` | — |
| 13 | `dequeue()` | Remove max element (6) $\Longrightarrow$ Queue: `[2, 1, 0]` | **6** |
| 14 | `dequeue()` | Remove max element (2) $\Longrightarrow$ Queue: `[1, 0]` | **2** |
| 15 | `enqueue(100)` | `[100, 1, 0]` | — |
| 16 | `dequeue()` | Remove max element (100) $\Longrightarrow$ Queue: `[1, 0]` | **100** |
| 17 | `dequeue()` | Remove max element (1) $\Longrightarrow$ Queue: `[0]` | **1** |

- **Final Queue State**: `[0]`
- **Sequence of Dequeued Values**: `20, 30, 4, 50, 6, 2, 100, 1`

**4(b) Stack Operations (10 Marks) —**

**Problem Statement**

Show the stack after each operation of the following series of stack operations, if executed upon an initially empty stack:

```
push(10), push(20), pop(), push(0), push(1), pop(), pop(),
push(2), push(30), pop(),
push(3), push(4), pop(), pop(), push(100), pop(), pop()
```

**Behavioral Rules of Stack**

- Last-In, First-Out (LIFO). Elements are pushed and popped strictly at the **top**.
- Mental Model: Like the nested call stack in recursive DNA sequence parsing or RNA secondary structure stem-loop matching.

**Step-by-Step Trace**

| Step | Operation | Stack State (Bottom → Top) | Top Element | Value Returned by [redacted] |
|---|---|---|---|---|
| 1 | `push(10)` | `[10]` | 10 | — |
| 2 | `push(20)` | `[10, 20]` | 20 | — |
| 3 | `pop()` | `[10]` | 10 | **20** |
| 4 | `push(0)` | `[10, 0]` | 0 | — |
| 5 | `push(1)` | `[10, 0, 1]` | 1 | — |
| 6 | `pop()` | `[10, 0]` | 0 | **1** |
| 7 | `pop()` | `[10]` | 10 | **0** |
| 8 | `push(2)` | `[10, 2]` | 2 | — |
| 9 | `push(30)` | `[10, 2, 30]` | 30 | — |
| 10 | `pop()` | `[10, 2]` | 2 | **30** |
| 11 | `push(3)` | `[10, 2, 3]` | 3 | — |
| 12 | `push(4)` | `[10, 2, 3, 4]` | 4 | — |
| 13 | `pop()` | `[10, 2, 3]` | 3 | **4** |
| 14 | `pop()` | `[10, 2]` | 2 | **3** |
| 15 | `push(100)` | `[10, 2, 100]` | 100 | — |
| 16 | `pop()` | `[10, 2]` | 2 | **100** |
| 17 | `pop()` | `[10]` | 10 | **2** |

- **Final Stack State**: `[10]` (Top is `10`)
- **Sequence of Popped Values**: `20, 1, 0, 30, 4, 3, 100, 2`

## Question 5: Queue Reversal (10 Marks)

**Problem Statement —** Given an integer $k$ and a queue of integers, implement a function to reverse the order of the first $k$ elements of the queue while keeping the other elements in the same original order.
**Allowed Queue Operations Only**: - `enqueue(x)`: Add item $x$ to rear of queue. - `dequeue()`: Remove item from front of queue. - `size()`: Returns number of elements. - `front()`: Returns front item without removing.

**Examples**

- **Input**: $q = [1, 2, 3, 4, 5], k = 3 \Longrightarrow$ **Output**: $[3, 2, 1, 4, 5]$
- **Input**: $q = [4, 3, 2, 1], k = 4 \Longrightarrow$ **Output**: $[1, 2, 3, 4]$

**Algorithm Design & The LIFO Inversion Principle —** A Queue is FIFO (First-In, First-Out), which preserves order. A Stack is LIFO (Last-In, First-Out), which **naturally reverses order** (just like taking the reverse complement of an oligonucleotide).
To reverse the first $k$ elements while preserving the remaining $n - k$ elements: 1. **Extract & Reverse**: Dequeue the first $k$ elements from the queue and push them onto an auxiliary Stack. 2. **Re-insert Reversed**: Pop all $k$ elements from the stack and enqueue them back into the queue. They are now at the *back* of the queue in reversed order. 3. **Rotate Remaining Elements**: The remaining $n - k$ elements are currently sitting at the front of the queue in their original order. Dequeue and immediately enqueue them one-by-one ($n - k$ times). This rotates them behind the reversed prefix, restoring their original relative order.
**Complete Java Implementation —**

```
import java.util.Queue;
import java.util.Stack;

public class QueueReversal {

    public static void reverseFirstK(Queue<Integer> q, int k) {
        // Validation / Edge Cases
        if (q == null || k <= 0 || k > q.size()) {
            return;
        }

        Stack<Integer> stack = new Stack<>();

        // Step 1: Dequeue first k elements and push onto stack
        for (int i = 0; i < k; i++) {
            stack.push(q.poll()); // dequeue
        }

        // Step 2: Pop from stack and enqueue back to the queue (reversed order)
        while (!stack.isEmpty()) {
            q.add(stack.pop()); // enqueue
        }

        // Step 3: Rotate the remaining (size - k) elements to the rear
        int remaining = q.size() - k;
        for (int i = 0; i < remaining; i++) {
            q.add(q.poll()); // dequeue front and enqueue back
        }
    }
}
```

**Complexity Analysis —**

- **Time Complexity**:
  – Pushing $k$ elements to stack: $\mathcal{O}(k)$

– Popping $k$ elements to queue: $\mathcal{O}(k)$
– Rotating $(n-k)$ elements: $\mathcal{O}(n-k)$
– Total Time: $\mathcal{O}(k+k+n-k) = \mathcal{O}(n)$, strictly linear.
• **Space Complexity**: $\mathcal{O}(k)$ auxiliary space for the auxiliary stack.

---

## SECTION 11: FULL MOCK EXAM 2 WALKTHROUGH & CODE

---

### Question 1: Stability & Best-Case Comparison (20 Marks)

• **1(a) Stability (10M)**: Preserves relative input order of duplicate keys. Insertion Sort is **STABLE** (strict `a[j] > key` stops shift on equal keys). Selection Sort is **UNSTABLE** (long-range swaps jump duplicates, e.g., $[2_a, 2_b, 1] \to [1, 2_b, 2_a]$). *(See Section 2.6.1 for full proof).*
• **1(b) Best-Case Already Sorted (10M)**:
– *Insertion Sort*: Outer loop runs $n-1$ times; inner loop terminates on 1st check ($a[i-1] \leq key$). Exactly $n-1$ comparisons, 0 shifts $\implies \mathbf{O(n)}$.
– *Selection Sort*: Non-adaptive. Inner loop unconditionally scans remaining suffix for minimum. Always $\sum(n-1-i) = \frac{n(n-1)}{2}$ comparisons, 0 (or $n-1$) swaps $\implies \mathbf{O(n^2)}$.

---

### Question 2: Sorting (15 Marks)

**Problem Statement —** Given the sequence $S$:

$$45, 12, 85, 32, 89, 45, 21$$

Provide the detailed steps to arrange $S$ in **ascending order** using the Selection Sort algorithm.
**Constraint**: In each iteration, you **MUST** find the **MAXIMUM** element and swap it with the element at the **END** of the unsorted portion (instead of the traditional way of finding the minimum and swapping it to the front). Show the changes in $S$ at each iteration.

---

**Step-by-Step Execution —**
• Array length: $n = 7$.
• Indices: `0` to `6`.
• Unsorted portion at iteration $k$ is $A[0 \ldots \text{end}]$, where end ranges from 6 down to 1.
• Tie-breaking convention for equal keys: We maintain the standard strict comparison (`if (A[j] > maxVal)`), keeping the first occurrence encountered.

| Iteration | Unsorted Range | Max Found (Value @ Index) | End Index | Swap Action | Array State After Iteration |
|---|---|---|---|---|---|
| **Initial** | `A[0..6]` | — | — | — | `[45, 12, 85, 32, 89, 45, 21]` |
| **1** | `A[0..6]` | **89** at index 4 | 6 | Swap $A[4]$ (89) with $A[6]$ (21) | `[45, 12, 85, 32, 21, 45, 89]` |
| **2** | `A[0..5]` | **85** at index 2 | 5 | Swap $A[2]$ (85) with $A[5]$ (45) | `[45, 12, 45, 32, 21, 85, 89]` |
| **3** | `A[0..4]` | **45** at index 0 | 4 | Swap $A[0]$ (45) with $A[4]$ (21) | `[21, 12, 45, 32, 45, 85, 89]` |
| **4** | `A[0..3]` | **45** at index 2 | 3 | Swap $A[2]$ (45) with $A[3]$ (32) | `[21, 12, 32, 45, 45, 85, 89]` |
| **5** | `A[0..2]` | **32** at index 2 | 2 | Max is already at index 2 (self-swap) | `[21, 12, 32, 45, 45, 85, 89]` |
| **6** | `A[0..1]` | **21** at index 0 | 1 | Swap $A[0]$ (21) with $A[1]$ (12) | `[12, 21, 32, 45, 45, 85, 89]` |

[!NOTE] Observe Iteration 3: The element `45` at index 0 was swapped with `21` at index 4, jumping past the second `45` at index 2! This vividly demonstrates in practice why Selection Sort is unstable.

**Final Result —**

$$[\mathbf{12}, \mathbf{21}, \mathbf{32}, \mathbf{45}, \mathbf{45}, \mathbf{85}, \mathbf{89}]$$

---

### Question 3: Linked List (35 Marks)

**Problem Statement —** Given the sorted (ascending order) `SinglyLinkList` class below:

```
public class SinglyLinkList {
    private Node first;

    public void moveLastToFront() {
        // YOUR CODE
    }

    public void removeDuplicates() {
        // YOUR CODE
    }
}

public class Node {
    private int key;
    private Node next;
}
```

• **a)** (15 marks) Complete the `moveLastToFront()` method that removes the last node of the list and makes it the new first node. (You must handle edge cases like an empty list or a list with only one node).
• **b)** (20 marks) Complete the `removeDuplicates()` method that removes all duplicate values from the sorted list, keeping only the first occurrence of each value.

---

**Solution 3(a): `moveLastToFront()` —**

**Edge Cases & Pointer Manipulation**

1. **Edge Case (0 or 1 Node)**: If `first == null` (empty list) or `first.next == null` (single node), moving the last to front changes nothing. Return immediately.
2. **General Case ($\geq$ 2 Nodes)**:
   • We need to locate the **second-to-last node** (`secLast`), because its `next` pointer must be set to `null` to detach the last node cleanly.
   • Traverse: `while (secLast.next.next != null) secLast = secLast.next;`.
   • Identify `last = secLast.next`.
   • Sever link: `secLast.next = null;`.
   • Prepend: `last.next = first;`.
   • Update head: `first = last;`.

**Java Implementation**

```
public void moveLastToFront() {
    // Edge case: Empty list or list with only one node
    if (first == null || first.next == null) {
        return;
    }

    // Traverse to find the second-to-last node
    Node secLast = first;
    while (secLast.next.next != null) {
        secLast = secLast.next;
    }

    // secLast.next is the last node
    Node last = secLast.next;

    // Disconnect the last node from the end
    secLast.next = null;

    // Attach last node before current first node
    last.next = first;

    // Make last node the new head
    first = last;
}
```

• **Time Complexity**: $\mathcal{O}(n)$ because we must traverse the list once to reach the second-to-last node.
• **Space Complexity**: $\mathcal{O}(1)$ auxiliary space.

---

**Solution 3(b): `removeDuplicates()` —**

**Algorithm & Pointer Manipulation**

Because the linked list is **already sorted in ascending order**, all identical values are contiguous (adjacent in memory). 1. If the list is empty (`first == null`), do nothing. 2. Maintain a pointer `current = first`. 3. While `current.next != null`: - If `current.key == current.next.key`, bypass the duplicate node: `java current.next = current.next.next;` *(Crucial: Do not advance `current` here, because the newly spliced `current.next` might also be a duplicate, e.g., in `1 -> 1 -> 1`).* - If `current.key != current.next.key`, advance: `java current = current.next;`

**Java Implementation**

```
public void removeDuplicates() {
    if (first == null) {
        return;
    }

    Node current = first;
    while (current.next != null) {
        if (current.key == current.next.key) {
            // Duplicate detected: bypass current.next
            current.next = current.next.next;
        } else {
            // Distinct value: advance to the next node
            current = current.next;
        }
    }
}
```

• **Time Complexity**: $\mathcal{O}(n)$ single linear pass.
• **Space Complexity**: $\mathcal{O}(1)$ in-place modification.

---

### Question 4: Queue and Stack (20 Marks)

**4(a) Min Priority Queue Operations (10 Marks) —**

**Problem Statement**

Consider a **MIN PRIORITY QUEUE**. Execute the following sequence of operations on an initially empty queue and write down the exact sequence of values returned by the `dequeue()` operations:

```
enqueue(15), enqueue(10), enqueue(20), dequeue(), enqueue(5), dequeue(),
dequeue(), enqueue(30), enqueue(2), dequeue()
```

**Behavioral Rules**

• `enqueue(x)`: Adds item $x$.
• `dequeue()`: Always removes and returns the **minimum numerical value** currently present in the queue.

**Step-by-Step Trace**

| Step | Operation | Priority Queue Contents (Sorted Ascending) | Value Returned by [illegible] |
|---|---|---|---|
| 1 | `enqueue(15)` | `[15]` | — |
| 2 | `enqueue(10)` | `[10, 15]` | — |
| 3 | `enqueue(20)` | `[10, 15, 20]` | — |
| 4 | `dequeue()` | Remove min (10) $\implies$ Queue: `[15, 20]` | **10** |
| 5 | `enqueue(5)` | `[5, 15, 20]` | — |
| 6 | `dequeue()` | Remove min (5) $\implies$ Queue: `[15, 20]` | **5** |
| 7 | `dequeue()` | Remove min (15) $\implies$ Queue: `[20]` | **15** |
| 8 | `enqueue(30)` | `[20, 30]` | — |
| 9 | `enqueue(2)` | `[2, 20, 30]` | — |
| 10 | `dequeue()` | Remove min (2) $\implies$ Queue: `[20, 30]` | **2** |

**Final Answer**

The exact sequence of values returned by the `dequeue()` operations is:

$$\mathbf{10}, \mathbf{5}, \mathbf{15}, \mathbf{2}$$

---

**4(b) Circular Queue Trace (10 Marks) —**

**Problem Statement**

Consider a Circular Queue implemented using an array $A$ of maximum capacity $N = 5$ (indices 0 to 4). Initially:

$$\text{front} = 0, \quad \text{rear} = -1, \quad \text{size} = 0$$

Show the exact contents of the array $A$, and the values of `front` and `rear` indices after executing the following sequence:

```
enqueue(10), enqueue(20), enqueue(30), dequeue(), enqueue(40),
enqueue(50), enqueue(60), dequeue()
```

**Formulas for Array-Based Circular Queue**

• enqueue($x$): $\text{rear} = (\text{rear} + 1) \pmod N$; $A[\text{rear}] = x$; $\text{size} = \text{size} + 1$.
• dequeue(): $x = A[\text{front}]$; $\text{front} = (\text{front} + 1) \pmod N$; $\text{size} = \text{size} - 1$.

**Step-by-Step Trace**

| Step | Operation | front | rear | size | Array $A$ Contents [illegible] | Explanation |
|---|---|---|---|---|---|---|
| **0** | Initial | 0 | −1 | 0 | `[null, null, null, null, null]` | Empty queue |
| **1** | `enqueue(10)` | 0 | 0 | 1 | `[10, null, null, null, null]` | rear $= (-1+1) \pmod 5 = 0$; store 10 |
| **2** | `enqueue(20)` | 0 | 1 | 2 | `[10, 20, null, null, null]` | rear $= (0+1) \pmod 5 = 1$; store 20 |
| **3** | `enqueue(30)` | 0 | 2 | 3 | `[10, 20, 30, null, null]` | rear $= (1+1) \pmod 5 = 2$; store 30 |
| **4** | `dequeue()` | 1 | 2 | 2 | `[10, 20, 30, null, null]` | Value 10 dequeued; front $= (0+1) \pmod 5 = 1$ |
| **5** | `enqueue(40)` | 1 | 3 | 3 | `[10, 20, 30, 40, null]` | rear $= (2+1) \pmod 5 = 3$; store 40 |
| **6** | `enqueue(50)` | 1 | 4 | 4 | `[10, 20, 30, 40, 50]` | rear $= (3+1) \pmod 5 = 4$; store 50 |
| **7** | `enqueue(60)` | 1 | **0** | 5 | `[60, 20, 30, 40, 50]` | rear **wraps around**: $(4+1) \pmod 5 = 0$; 60 overwrites index 0 |
| **8** | `dequeue()` | **2** | **0** | 4 | `[60, 20, 30, 40, 50]` | Value 20 dequeued; front $= (1+1) \pmod 5 = 2$ |

**Final State**

• **Array $A$ Contents**: `[60, 20, 30, 40, 50]` *(In memory, index 1 still contains 20 unless cleared, but logically the queue contains indices 2, 3, 4, 0).*
• **Active Elements (Front to Rear)**: `30, 40, 50, 60`
• **front Index**: `2`
• **rear Index**: `0`
• **size**: `4`

---

### Question 5: Stack & Queue (10 Marks)

**Problem Statement —** Given a stack of integers, implement a function to sort the stack in **ascending order** (smallest items on top).
**Constraint**: You are allowed to use **exactly one additional auxiliary Stack**. You may **NOT** use arrays, queues, lists, or any other data structures.
**Only the following operations are allowed on a stack**: - `push(x)`: Add an item $x$ to the top. - `pop()`: Remove and return the top item. - `peek()`: Return the top item without removing it. - `isEmpty()`: Check if the stack is empty.

**Example**

• **Input Stack (Top is right)**: `[34, 3, 31, 98, 92, 23]` (23 on top)
• **Output Stack (Top is right)**: `[98, 92, 34, 31, 23, 3]` (3 on top)

---

**Algorithm Walkthrough —** Let the given stack be `input` and our auxiliary stack be `aux`.

**The Inversion Principle**

If we want the final `input` stack to have the **smallest items on top** (ascending from top down, meaning largest at the bottom), our intermediate `aux` stack must hold elements in **descending order** (largest items on top). When we finally pour `aux` back into `input`, the order inverts, placing the smallest items on top!

**Step-by-Step Procedure**

1. While `input` is not empty:

- Pop the top element from `input` and store it in a temporary variable: `tmp = input.pop()`.
- While `aux` is not empty AND `aux.peek() > tmp`:
  - Pop from `aux` and push it back into `input`: `input.push(aux.pop())`. *(This shifts larger elements out of the way, transferring them temporarily back to `input`).*
- Push `tmp` onto `aux`.

2. Once `input` is empty, all elements are in `aux` sorted with the largest element at the top.
3. Transfer all elements from `aux` back to `input` (`input.push(aux.pop())`).
4. `input` now contains all elements sorted with the **smallest item on top**.

**Complete Java Implementation —**

```
import java.util.Stack;

public class StackSorter {

    public static void sortStack(Stack<Integer> input) {
        if (input == null || input.isEmpty()) {
            return;
        }

        Stack<Integer> aux = new Stack<>();

        // Process each element from the input stack
        while (!input.isEmpty()) {
            int tmp = input.pop();

            // Shift elements in aux that are greater than tmp back into input
            while (!aux.isEmpty() && aux.peek() > tmp) {
                input.push(aux.pop());
            }

            // Place tmp in its correct sorted position in aux
            aux.push(tmp);
        }

        // Transfer elements back to input stack:
        // aux has largest on top -> pouring into input puts smallest on top!
        while (!aux.isEmpty()) {
            input.push(aux.pop());
        }
    }
}
```

**Complexity Analysis —**

- **Time Complexity**: $\mathcal{O}(n^2)$ in the worst case (e.g., if the input stack is already sorted in reverse order, each insertion shifts elements back and forth between the two stacks, identical to Insertion Sort mechanics).
- **Space Complexity**: $\mathcal{O}(n)$ auxiliary space to hold the $n$ elements in the single additional `aux` stack.

---

## SECTION 12: FULL REAL EXAM 2023 (IT013IU) WALK-THROUGH & CODE

---

### Question 1: Sorting (20 Marks)

**Problem Statement —** Given array $A$:

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Data | 72 | 4 | 12 | 53 | 15 | 2 | 8 | 16 | 40 | 21 | 7 | 87 | 35 |

Sort the array with **Selection Sort** by filling the table below.

---

**Step-by-Step Selection Sort Execution Table —**

| Iteration / Action | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Initial Array** | 72 | 4 | 12 | 53 | 15 | 2 | 8 | 16 | 40 | 21 | 7 | 87 | 35 |
| **Step 1** (Min = 2 @ idx 5 ↔ Swap $A[0]$) | **2** | 4 | 12 | 53 | 15 | 72 | 8 | 16 | 40 | 21 | 7 | 87 | 35 |
| **Step 2** (Min = 4 @ idx 1 ↔ Self-swap $A[1]$) | **2** | **4** | 12 | 53 | 15 | 72 | 8 | 16 | 40 | 21 | 7 | 87 | 35 |
| **Step 3** (Min = 7 @ idx 10 ↔ Swap $A[2]$) | **2** | **4** | **7** | 53 | 15 | 72 | 8 | 16 | 40 | 21 | 12 | 87 | 35 |
| **Step 4** (Min = 8 @ idx 6 ↔ Swap $A[3]$) | **2** | **4** | **7** | **8** | 15 | 72 | 53 | 16 | 40 | 21 | 12 | 87 | 35 |
| **Step 5** (Min = 12 @ idx 10 ↔ Swap $A[4]$) | **2** | **4** | **7** | **8** | **12** | 72 | 53 | 16 | 40 | 21 | 15 | 87 | 35 |
| **Step 6** (Min = 15 @ idx 10 ↔ Swap $A[5]$) | **2** | **4** | **7** | **8** | **12** | **15** | 53 | 16 | 40 | 21 | 72 | 87 | 35 |
| **Step 7** (Min = 16 @ idx 7 ↔ Swap $A[6]$) | **2** | **4** | **7** | **8** | **12** | **15** | **16** | 53 | 40 | 21 | 72 | 87 | 35 |
| **Step 8** (Min = 21 @ idx 9 ↔ Swap $A[7]$) | **2** | **4** | **7** | **8** | **12** | **15** | **16** | **21** | 40 | 53 | 72 | 87 | 35 |
| **Step 9** (Min = 35 @ idx 12 ↔ Swap $A[8]$) | **2** | **4** | **7** | **8** | **12** | **15** | **16** | **21** | **35** | 53 | 72 | 87 | 40 |
| **Step 10** (Min = 40 @ idx 12 ↔ Swap $A[9]$) | **2** | **4** | **7** | **8** | **12** | **15** | **16** | **21** | **35** | **40** | 72 | 87 | 53 |
| **Step 11** (Min = 53 @ idx 12 ↔ Swap $A[10]$) | **2** | **4** | **7** | **8** | **12** | **15** | **16** | **21** | **35** | **40** | **53** | 87 | 72 |
| **Step 12** (Min = 72 @ idx 12 ↔ Swap $A[11]$) | **2** | **4** | **7** | **8** | **12** | **15** | **16** | **21** | **35** | **40** | **53** | **72** | **87** |

---

### Question 2: Queue and Stack (20 Marks)

#### 2(a) FIFO Queue Trace (10 Marks) —

**Problem Statement**

What values are returned after each `dequeue()` during the following sequence of queue operations, if executed on an initially empty queue?

```
enqueue(0), enqueue(3), dequeue(), enqueue(0), enqueue(4),
dequeue(), dequeue(),
enqueue(2), enqueue(0), dequeue(), enqueue(2), enqueue(3),
dequeue(), dequeue(),
enqueue(1330), dequeue(), dequeue()
```

**Step-by-Step Trace**

| Step | Operation | Queue State (Front → Rear) | Returned by [redacted] |
|---|---|---|---|
| 1 | `enqueue(0)` | `[0]` | — |
| 2 | `enqueue(3)` | `[0, 3]` | — |
| 3 | `dequeue()` | `[3]` | **0** |
| 4 | `enqueue(0)` | `[3, 0]` | — |
| 5 | `enqueue(4)` | `[3, 0, 4]` | — |
| 6 | `dequeue()` | `[0, 4]` | **3** |
| 7 | `dequeue()` | `[4]` | **0** |
| 8 | `enqueue(2)` | `[4, 2]` | — |
| 9 | `enqueue(0)` | `[4, 2, 0]` | — |
| 10 | `dequeue()` | `[2, 0]` | **4** |
| 11 | `enqueue(2)` | `[2, 0, 2]` | — |
| 12 | `enqueue(3)` | `[2, 0, 2, 3]` | — |
| 13 | `dequeue()` | `[0, 2, 3]` | **2** |
| 14 | `dequeue()` | `[2, 3]` | **0** |
| 15 | `enqueue(1330)` | `[2, 3, 1330]` | — |
| 16 | `dequeue()` | `[3, 1330]` | **2** |
| 17 | `dequeue()` | `[1330]` | **3** |

**Final Sequence of Values Returned by `dequeue()`**

$$0, 3, 0, 4, 2, 0, 2, 3$$

*(Notice the hidden easter egg: `03, 04, 2023` encodes the exact date of the examination: April 3, 2023!)*

---

#### 2(b) LIFO Stack Trace (10 Marks) —

**Problem Statement**

What values are returned after each `pop()` during the following series of stack operations, if executed upon an initially empty stack?

```
push(0), push(3), pop(), push(0), push(4), pop(), pop(), push(2),
push(0), pop(),
push(2), push(3), pop(), pop(), push(1330), pop(), pop()
```

**Step-by-Step Trace**

| Step | Operation | Stack State (Bottom → Top) | Top Element | Returned by [redacted] |
|---|---|---|---|---|
| 1 | `push(0)` | `[0]` | 0 | — |
| 2 | `push(3)` | `[0, 3]` | 3 | — |
| 3 | `pop()` | `[0]` | 0 | **3** |
| 4 | `push(0)` | `[0, 0]` | 0 | — |
| 5 | `push(4)` | `[0, 0, 4]` | 4 | — |
| 6 | `pop()` | `[0, 0]` | 0 | **4** |
| 7 | `pop()` | `[0]` | 0 | **0** |
| 8 | `push(2)` | `[0, 2]` | 2 | — |
| 9 | `push(0)` | `[0, 2, 0]` | 0 | — |
| 10 | `pop()` | `[0, 2]` | 2 | **0** |
| 11 | `push(2)` | `[0, 2, 2]` | 2 | — |
| 12 | `push(3)` | `[0, 2, 2, 3]` | 3 | — |
| 13 | `pop()` | `[0, 2, 2]` | 2 | **3** |
| 14 | `pop()` | `[0, 2]` | 2 | **2** |
| 15 | `push(1330)` | `[0, 2, 1330]` | 1330 | — |
| 16 | `pop()` | `[0, 2]` | 2 | **1330** |
| 17 | `pop()` | `[0]` | 0 | **2** |

**Final Sequence of Values Returned by `pop()`**

$$3, 4, 0, 0, 3, 2, 1330, 2$$

---

### Question 3: Linked List (30 Marks)

**Problem Statement —** Given the 2 classes `Node` and `DoublyLinkList`:

```
public class DoublyLinkList { public Node first, last; }
public class Node { public int key; public Node next, previous; }
```

- **(5 marks)** Implement function `delete_first` for the doubly linked list.
- **(5 marks)** Implement function `insert_last` for the doubly linked list.
- **(20 marks)** Implement an algorithm to merge two sorted doubly linked lists to make a new sorted linked list.

---

#### 3.1 Function `delete_first()` (5 Marks) —

**Logic & Edge Cases**

- **Empty List**: If `first == null`, nothing to delete.
- **Single Node** (`first == last`): Set both `first = null` and `last = null`.
- **Multiple Nodes**: Advance `first = first.next`, and sever the reverse link by setting `first.previous = null`.

```
public void delete_first() {
    if (first == null) return;
    if (first == last) { first = null; last = null; }
    else { first = first.next; first.previous = null; }
}
```

- **Time Complexity**: $\mathcal{O}(1)$ constant time.
- **Space Complexity**: $\mathcal{O}(1)$.

---

#### 3.2 Function `insert_last(int key)` (5 Marks) —

**Logic & Edge Cases**

- Create a `newNode` with `newNode.key = key`.
- **Empty List** (`first == null`): Both `first` and `last` point to `newNode`.
- **Non-Empty List**:
  - `last.next = newNode;`
  - `newNode.previous = last;`
  - `last = newNode;`

```
public void insert_last(int key) {
    Node newNode = new Node();
    newNode.key = key;
    newNode.next = null;
    newNode.previous = null;

    if (first == null) {
        // List was empty
        first = newNode;
        last = newNode;
    } else {
        // Attach to current last and update last pointer
        last.next = newNode;
        newNode.previous = last;
        last = newNode;
    }
}
```

- **Time Complexity**: $\mathcal{O}(1)$ constant time (since a direct `last` pointer is maintained).
- **Space Complexity**: $\mathcal{O}(1)$.

---

#### 3.3 Algorithm to Merge Two Sorted Doubly Linked Lists (20 Marks) —

**Design Decision: Non-Destructive vs. In-Place Splicing**

The problem asks to *"make a new sorted linked list"*. Creating a clean `result` list and appending via `insert_last` preserves the integrity of the original lists and eliminates brittle pointer overwriting bugs.

**Java Implementation**

```
public static DoublyLinkList merge(DoublyLinkList list1,
DoublyLinkList list2) {
    DoublyLinkList result = new DoublyLinkList();

    // Guard against null list objects
    if (list1 == null && list2 == null) return result;
    if (list1 == null || list1.first == null) return list2;
    if (list2 == null || list2.first == null) return list1;

    Node p1 = list1.first;
    Node p2 = list2.first;

    // Two-pointer merge: compare current nodes from both lists
    while (p1 != null && p2 != null) {
        if (p1.key <= p2.key) {
            result.insert_last(p1.key);
            p1 = p1.next;
        } else {
            result.insert_last(p2.key);
            p2 = p2.next;
        }
    }

    // Append any remaining elements from list1
    while (p1 != null) {
        result.insert_last(p1.key);
        p1 = p1.next;
    }

    // Append any remaining elements from list2
    while (p2 != null) {
```

```
        result.insert_last(p2.key);
        p2 = p2.next;
    }

    return result;
}
```

- **Time Complexity**: $\mathcal{O}(n_1 + n_2)$, strictly linear in the total number of nodes. Each node is inspected once.
- **Space Complexity**: $\mathcal{O}(n_1 + n_2)$ for the newly created merged list.

---

## Question 4: Complexity (20 Marks)

**Problem Statement —** Propose the **worst-case complexity** of the following operations in linked-lists:

| Operation | Unsorted, simple (singly) linked list | Sorted simple (singly) linked list | Unsorted, double (doubly) linked list | Sorted double (doubly) linked list |
|---|---|---|---|---|
| **Search** | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ |
| **Insert a new value** | $\mathcal{O}(1)$ | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ | $\mathcal{O}(n)$ |
| **Delete a given value** | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ |
| **Get minimum** | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ |
| **Get maximum** | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ *(or $\mathcal{O}(1)$ with tail)* | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ |

---

### In-Depth Explanations & Common Exam Pitfalls —

**1. Search ($\mathcal{O}(n)$ across all four)**

- Linked lists are non-contiguous in memory. Unlike arrays, you cannot calculate memory addresses via index arithmetic ( `arr[i] = base + i * size` ). You cannot jump directly to the middle to perform Binary Search.
- To search for a key, you must follow pointers sequentially from the start.
- Even if the list is sorted, in the worst case (the target element is at the very end or not present at all), you must traverse all $n$ nodes. Hence, all four are $\mathcal{O}(n)$.

**2. Insert a new value**

- **Unsorted Lists ($\mathcal{O}(1)$)**: Because there is no ordering constraint, a new node can simply be inserted at the head of the list in $\mathcal{O}(1)$ constant time (or at the tail in $\mathcal{O}(1)$ if a tail pointer is maintained).
- **Sorted Lists ($\mathcal{O}(n)$)**: To maintain the sorted invariant, you **must first search** for the correct location where the value belongs. Scanning to the correct slot takes $\mathcal{O}(n)$ in the worst case. Once the position is found, pointer rewiring takes $\mathcal{O}(1)$, but the total insertion time is dominated by the search: $\mathcal{O}(n) + \mathcal{O}(1) = \boldsymbol{\mathcal{O}(n)}$. > [!WARNING] > **Exam Trap Alert**: Students often mistakenly write $\mathcal{O}(1)$ for sorted list insertion thinking only of pointer splicing. But in a sorted list, you cannot splice until you find *where* to splice, which requires an $\mathcal{O}(n)$ linear scan!

**3. Delete a given value ($\mathcal{O}(n)$ across all four)**

- The prompt specifies *"Delete a given value"*, not *"Delete a node given its pointer"*.
- To delete a given value, the algorithm must first traverse the list to locate the node containing that value: this search step takes $\mathcal{O}(n)$.
- In a singly linked list, you also need to find the predecessor node to bridge over the deleted node.
- In a doubly linked list, once the node is located, unlinking takes $\mathcal{O}(1)$ using `node.previous` and `node.next` . However, finding that node still took $\mathcal{O}(n)$ search steps. Total: $\boldsymbol{\mathcal{O}(n)}$.

**4. Get minimum**

- **Unsorted Lists ($\mathcal{O}(n)$)**: The minimum could be anywhere in the list; you must examine all $n$ nodes to be certain.
- **Sorted Lists ($\mathcal{O}(1)$)**: In ascending order, the minimum is always guaranteed to be the very first node ( `first.key` ). Reading it takes $\boldsymbol{\mathcal{O}(1)}$ immediate time.

**5. Get maximum**

- **Unsorted Lists ($\mathcal{O}(n)$)**: Unpredictable position; requires scanning all $n$ nodes.
- **Sorted Simple (Singly) List**:
  - Without a tail pointer: Must traverse the entire chain of $n$ nodes from `first` to reach the end: $\boldsymbol{\mathcal{O}(n)}$.
  - With a tail pointer: If the class maintains a `tail` pointer, accessing `tail.key` is $\boldsymbol{\mathcal{O}(1)}$.
- **Sorted Double (Doubly) List ($\mathcal{O}(1)$)**: As defined in the exam ( `DoublyLinkList` maintains `public Node last;` ), the maximum is immediately accessible via `last.key` in $\boldsymbol{\mathcal{O}(1)}$ time.

---

## Question 5: Queue (10 Marks)

**Problem Statement —** Whereas a stack allows insertion and deletion of elements at only one end, and a queue allows insertion at one end and deletion at the other end, a **deque** (double ended queue, pronounced like "deck") allows insertion and deletion at both ends.
Write **04 (four) $\mathcal{O}(1)$-time procedures** to insert elements into and delete elements from both ends of a deque implemented by an array.

---

**Conceptual Architecture of an Array-Based Deque —** We use a circular array `arr[]` of fixed capacity $C$, with variables: - `front` : Index of the front element (initially $-1$). - `rear` : Index of the rear element (initially $-1$). - `size` : Current count of stored elements (initially $0$).

**Index Movement Rules**

- **Moving Forward (Clocks-wise)**: `(idx + 1) % capacity`
- **Moving Backward (Counter-clockwise)**: `(idx - 1 + capacity) % capacity` *(Adding `capacity` before modulo ensures the result is never negative in Java)*.

---

**The 4 Required $\mathcal{O}(1)$ Procedures —**

```
public class ArrayDeque {
    private int[] arr;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public ArrayDeque(int capacity) {
        this.capacity = capacity;
        this.arr = new int[capacity];
        this.front = -1;
        this.rear = -1;
        this.size = 0;
    }

    public boolean isFull() {
        return size == capacity;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    // 1. insertFront: Insert element at the front end - O(1)
    public void insertFront(int key) {
        if (isFull()) {
            throw new RuntimeException("Deque Overflow: cannot
insert into full deque");
        }
        if (isEmpty()) {
            front = 0;
            rear = 0;
        } else {
            // Decrement front circularly
            front = (front - 1 + capacity) % capacity;
        }
        arr[front] = key;
        size++;
    }

    // 2. insertRear: Insert element at the rear end - O(1)
    public void insertRear(int key) {
        if (isFull()) {
            throw new RuntimeException("Deque Overflow: cannot
insert into full deque");
        }
        if (isEmpty()) {
            front = 0;
            rear = 0;
        } else {
            // Increment rear circularly
            rear = (rear + 1) % capacity;
        }
        arr[rear] = key;
        size++;
    }

    // 3. deleteFront: Delete and return element from front end -
O(1)
    public int deleteFront() {
        if (isEmpty()) {
            throw new RuntimeException("Deque Underflow: cannot
delete from empty deque");
        }
        int removedValue = arr[front];
        if (front == rear) {
            // Only one element was in the deque; reset to empty
            front = -1;
            rear = -1;
        } else {
            // Increment front circularly
            front = (front + 1) % capacity;
        }
        size--;
        return removedValue;
    }

    // 4. deleteRear: Delete and return element from rear end -
O(1)
    public int deleteRear() {
        if (isEmpty()) {
            throw new RuntimeException("Deque Underflow: cannot
delete from empty deque");
        }
        int removedValue = arr[rear];
        if (front == rear) {
            // Only one element was in the deque; reset to empty
            front = -1;
            rear = -1;
        } else {
            // Decrement rear circularly
            rear = (rear - 1 + capacity) % capacity;
        }
        size--;
        return removedValue;
    }
}
```

**Why Each Procedure is Strictly $\mathcal{O}(1)$ —**

- No elements are shifted or relocated in memory.
- Index adjustments use single arithmetic instructions ( `+` , `-` , `%` ).
- Memory access is direct through array index addressing ( `arr[front]` or `arr[rear]` ).
- Time complexity is unconditionally $\mathcal{O}(1)$ in best, average, and worst cases.

---

# SECTION 13: LAST-MINUTE EXAM QUICK-CHECK & COMMON TRAPS

## 13.1 Universal Defensive Edge-Case Checklist

Before submitting any code or tracing answer, check: 1. **Empty Structure**: What happens if `first == null` , `size == 0` , or `arr.length == 0` ? Always add an early return guard. 2. **Single Element**: Does the method work for 1 node ( `first.next == null` or `first == last` )? Updating `last` or `first` requires special care when length is 1. 3. **Circular Indexing**: Forward: `(idx + 1) % cap` , Backward: `(idx - 1 + cap) % cap` (prevents negative index). 4. **Midpoint Calculation**: Always use `low + (high - low) / 2` to prevent 32-bit signed integer overflow. 5. **Linked List Deletions**: Never advance your traversal pointer `cur` immediately after unlinking a duplicate node ( `cur.next = cur.next.next` ), because the newly attached node might also be a duplicate!

## 13.2 Rapid Mental Math & Recurrence Formulas

- Comparisons in Selection Sort: Exactly $\frac{n(n-1)}{2} \approx 0.5n^2$.
- Comparisons in Insertion Sort (Average): $\approx \frac{n^2}{4} = 0.25n^2$.
- Number of Swaps in Selection Sort: At most $n - 1$.
- Number of Shifts in Insertion Sort: At most $\frac{n(n-1)}{2}$, average $\approx \frac{n^2}{4}$.
- Tree Height (Heap / Balanced Tree): $\lfloor \log_2 n \rfloor$.
- Exam Rounding Convention: Ceiling division `(a + b - 1) / b` (e.g. $7/2 \Longrightarrow 4$).

## 13.3 Standard Input Scanner & Fast Output Template

```
import java.util.Scanner;
public class ExamInputTemplate {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt(), arr[] = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.println("Read " + n + " elements.");
        sc.close();
    }
}
```

## 13.4 Fundamental Recursion Templates (Call Stack Mechanics)

```
public class RecursionTemplates {
    // 1. Array Sum: O(n) time, O(n) stack space
    public static int sumArray(int[] a, int idx) {
        return (idx == a.length) ? 0 : a[idx] + sumArray(a, idx +
1);
    }
    // 2. Power function x^n: O(log n) time
    public static long fastPower(long x, int n) {
        if (n == 0) return 1;
        long half = fastPower(x, n / 2);
        return (n % 2 == 0) ? half * half : x * half * half;
    }
    // 3. Tower of Hanoi: O(2^n) time (move n disks from peg to
dest via aux)
    public static void hanoi(int n, char from, char to, char aux)
{
        if (n == 1) { System.out.println("Move 1: " + from + "->"
+ to); return; }
        hanoi(n - 1, from, aux, to);
        System.out.println("Move " + n + ": " + from + "->" + to);
        hanoi(n - 1, aux, to, from);
    }
}
```

## 13.5 Infix to Postfix Precedence Table & Stack Evaluation

- **Precedence**: `^` (exponent, right-to-left) > `*` , `/` (left-to-right) > `+` , `-` (left-to-right).
- **Rules**: Operands → output immediately. `(` → push. `)` → pop & output until `(` is popped. Operator → pop & output while stack not empty and `prec(peek) >= prec(op)` , then push operator. End → pop all remaining operators.